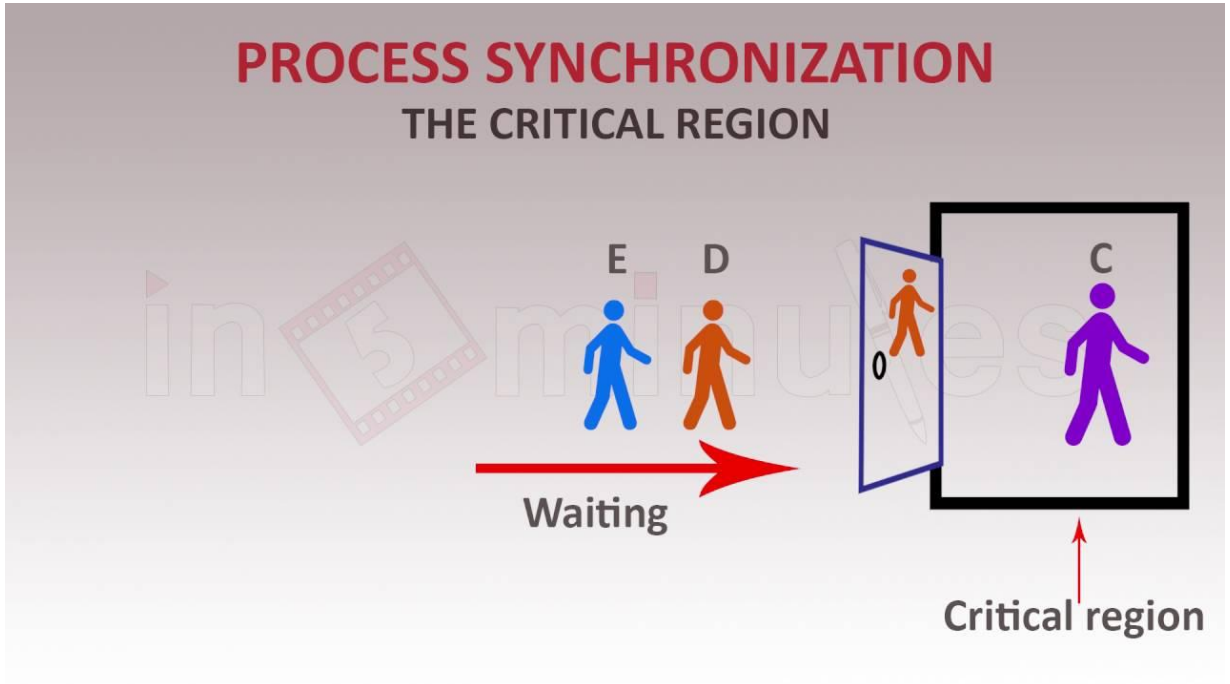
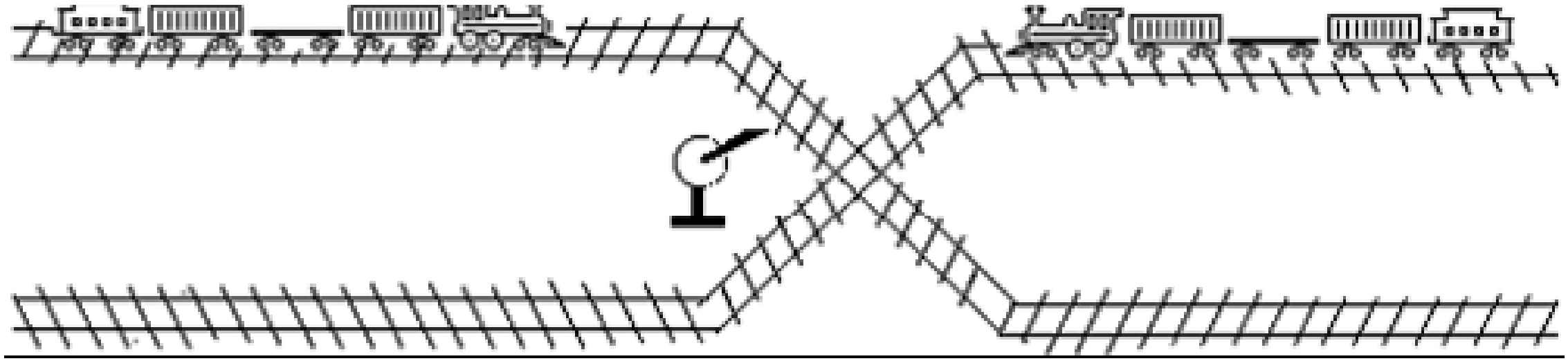
İşletim Sistemleri
(Operating Systems)



PROCESS SENKRONİZASYONU

(PROCESS SYNCHRONIZATION)

Semaforlar (Semaphores)



Semaforlar (Semaphores)

- 1965 yılında Edsger W. Dijkstra, process'ler arasındaki senkronizasyonu sağlamak için semafor yapısını geliştirmiştir.

- Semafor, işletim sistemi tarafından tanınan, donanım + yazılım desteği olan ve tamsayı (integer) değerlerini alabilen özel değişkenlerdir.

- *Semafor değeri, yalnızca bir process'in herhangi bir zamanda erişebileceği ve değiştirebileceği paylaşılan bir değerdir.*

Semaforlar (Semaphores)

- Semafor tamsayı değer saklayan bir nesne olarak düşünülebilir.
- Semafor üzerinde temel olarak **DOWN =P(S)** ve **UP =V(S)** isimli iki tür işlem tanımlıdır:

➢ P(S) semaforun değerini azaltmak için kullanılır.

➢ V(S) semaforun değerini arttırmak için kullanılır.

- Bu işlemler bölünemez (atomic) işlemlerdir.

Semaforlar (Semaphores)

- Dijkstra çalışmasında, Almanca ‘Proberen : test et’ manasına gelen **P** operatörünü ve ‘Verhogen : arttır’ manasına gelen **V** operatörünü tamsayı tipinde değişken olan ***‘S’*** semaforu üzerinde uygulamıştır.

P(S) ve V(S) operatörlerinin eşdeğer kodları şu şekilde olabilir:

semaphore S;

P(S) : **while *S*≤ 0 do {bekle}; S = S-1**

V(S) : **S = S+1**

- P(S) atomik biçimde S değişkenini sınayan eğer S sıfırdan küçük veya eşitse beklemeyi sağlayan, değilse içeriğini bir eksilten, bir işlemdir.
- V(S) ise S değişkenini bir artırmayı sağlayan bir işlemdir.

Semaforlar (Semaphores)

- ***'S'*** semaforu bir tamsayıdır ve sadece wait() ve signal() atomik komutları tarafından erişilebilir ve değiştirilebilir.

- wait(S) komutu P(S) yerine, signal(S) komutu V(S) yerine kullanılır.

- wait() semaforun değerini bir azaltır.
- signal() semaforun değerini bir arttırır.

- *Bu iki yol dışında semaforun değerini değiştirmek mümkün değildir.*

Semaforlar (Semaphores)

- Bir process kritik bölgesine girmeden önce wait(S) komutunu çalıştırır.
- S değişkeninin içerdiği değere göre kritik bölgesine girer yada wait() komutu üzerinde bekler.
- Kritik bölge çıkışında mutlaka signal(S) komutunun çalıştırılması gerekir.

Semaforlar (Semaphores)

- wait() ve signal() bölünemez (atomik) olarak gerçekleştirilirler.

- Bu sebeple wait() ve signal() içinde process'lere müdahale edilmesi ve CPU'nun kritik bölgesini yürütecek bir başka process'e verilmesi söz konusu değildir.

- Process'ler kritik bölgelerine girmeden wait(), kritik bölgelerinden çıkarken de signal() işlemlerini yaparak karşılıklı dışlamayı (mutual exclusion) garanti ederler.

Semaforlar (Semaphores)

- Semaforlar kullanım alanları ve tasarımları itibariyle ikiye ayrılır:
  - İkili Semafor (Binary Semaphore)
  - Sayma Semaforu (Counting Semaphore)

- İkili semafor değişkeni 0 ve 1 olmak üzere sadece iki değer alabilir. Bu, bir kaynağın ya tamamen kilitli (0) ya da serbest (1) olduğunu gösterir. Semafor değeri 1 olarak başlatılır. Birden fazla process ve tek bir kaynakla kritik bölüm problemlerinin çözümünü uygulamak için kullanılır.
- Sayma semaforları ise herhangi bir tamsayı değeri alabilirler. Sayma semaforları, birden fazla sayıda örnekten (instance) oluşan bir kaynağa erişimi düzenlemek için kullanılabilir.
- İkili semaforlar basit bir "açık/kapalı" mekanizması sunarken, sayma semaforları daha karmaşık ve çoklu kaynak yönetimini destekler.

İkili semaforlar (Binary semaphores)
wait() ve signal() komut eşdeğer yapısı

```
waitB(S):

if (S.value = 1)
{
  S.value := 0;
}
  else
{
Bu process'i bloke durumuna al
Bu process'i Semafor kuyruğuna
yerleştir
}
```

BLOCK

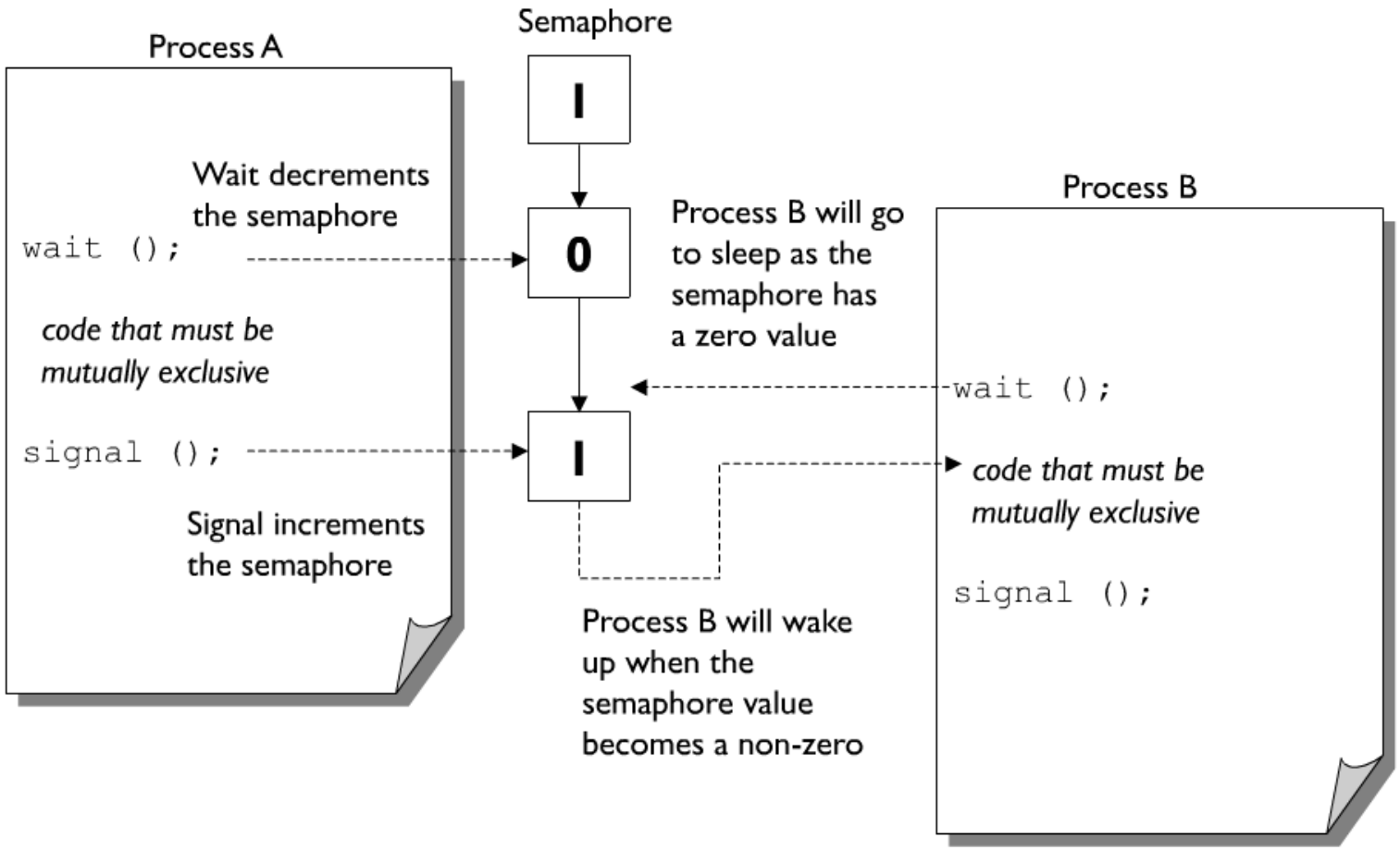
```
signalB(S):

if (S.queue is empty)
{
  S.value := 1;
}
  else
{
Semafor kuyruğundan bir P
process'ini çıkar
Bu P process'ini hazır (ready)
kuyruğuna yerleştir
}
```

WAKE UP

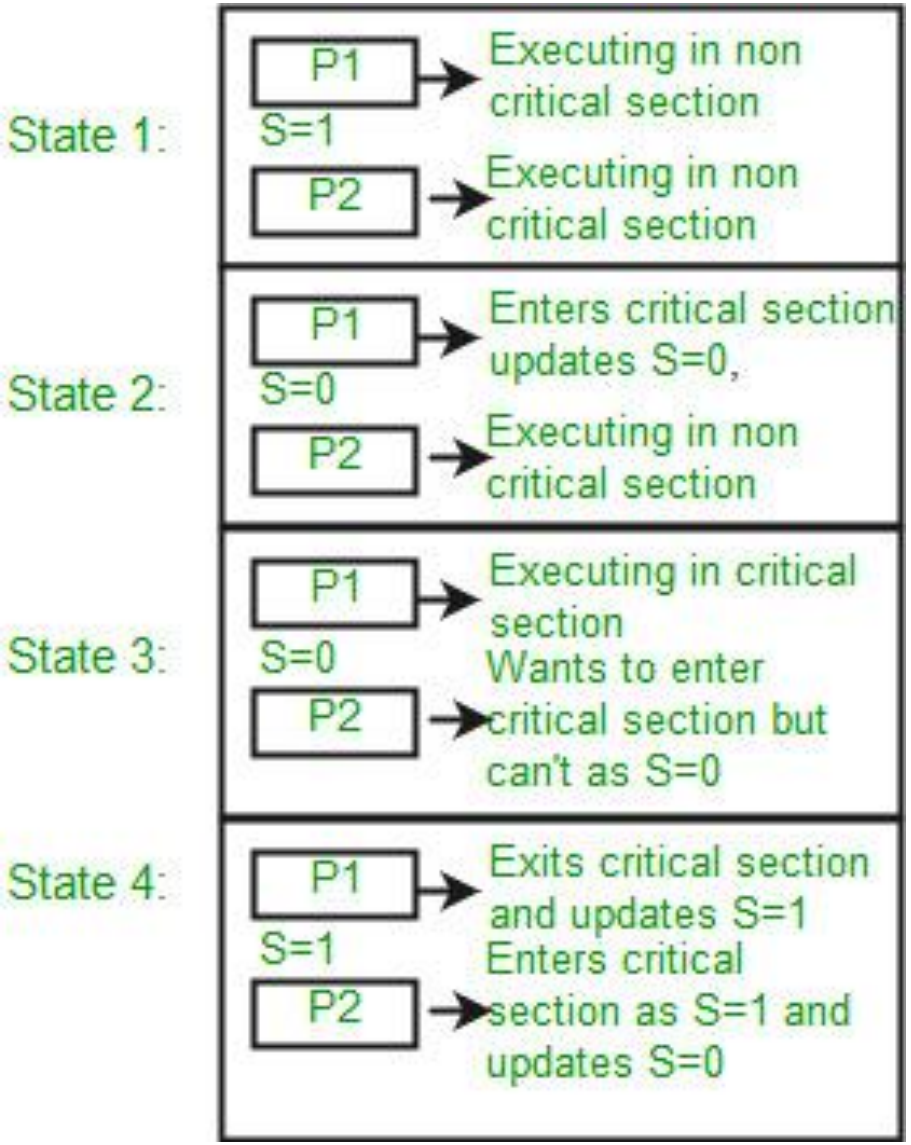
İkili (binary) semafor değerleri yalnızca 0 veya 1 olabilir. Başlangıç değeri, genellikle 1 olarak ayarlanır. 0 değeri, kaynağın şu anda kullanılmadığını, 1 değeri ise kaynağın kullanıldığını gösterir.

İkili semaforlar (Binary semaphores)

P (wait): Semaforun değerini kontrol eder. Eğer değer 1 ise, değeri 0'a düşürür ve kaynağı kullanmaya başlar. Eğer değer 0 ise, process beklemeye alınır (bloke olur).
V (signal): Semaforun değerini 1'e yükseltir. Eğer bekleyen process varsa, uyandırılır ve kaynağı kullanmaya başlar.



İkili semaforlar (Binary semaphores)



İkili semaforlar (Binary semaphores)

Burada paylaşılan bir değişken `s` üzerinde dört process (P1, P2, P3, P4) işlem yapıyor. İşlemler `test(s)` (semaforu kontrol et ve kilitle) ve `increment(s)` (semafor değerini artır ve kilidi serbest bırak) şeklinde.

State Number	Actions Calling Process	Operation	Running in Critical Region	Results Blocked on *s*	Value of *s*
0					1
1	P1	test(*s*)	P1		0
2	P1	increment(*s*)			1
3	P2	test(*s*)	P2		0
4	P3	test(*s*)	P2	P3	0
5	P4	test(*s*)	P2	P3, P4	0
6	P2	increment(*s*)	P3	P4	0
7			P3	P4	0
8	P3	increment(*s*)	P4		0
9	P4	increment(*s*)			1

İkili semaforlar (Binary semaphores)

test(s) = wait(S):
if (s = 1)
{ s := 0; }
else
{Bu process'i bloke durumuna al ve process'i Semafor kuyruğuna yerleştir }

increment(s) = signal(S):
if (semafor kuyruğu boş ise)
{ s := 1; }
else
{Semafor kuyruğundan bir P process'ini çıkar. Bu P process'ini hazır (ready) kuyruğuna yerleştir.

State Number	Actions Calling Process	Operation	Running in Critical Region	Results Blocked on *s*	Value of *s*
0					1
1	P1	test(*s*)	P1		0
2	P1	increment(*s*)			1
3	P2	test(*s*)	P2		0
4	P3	test(*s*)	P2	P3	0
5	P4	test(*s*)	P2	P3, P4	0
6	P2	increment(*s*)	P3	P4	0
7			P3	P4	0
8	P3	increment(*s*)	P4		0
9	P4	increment(*s*)			1

İkili semaforlar (Binary semaphores)

Tablo'daki 5. durumdan sonra, en uzun bekleme süreci olan P3 process'i, kritik bölgeye girmek için seçilmişti, ancak sistem ilk giren ilk çıkar seçim politikasını kullanmıyorsa bu zorunlu değildir.

Aslında, yürütülecek process'in seçimi, Process Zamanlayıcısı tarafından kullanılan algoritmaya bağlıdır.

State Number	Actions Calling Process	Operation	Running in Critical Region	Results Blocked on *s*	Value of *s*
0					1
1	P1	test(*s*)	P1		0
2	P1	increment(*s*)			1
3	P2	test(*s*)	P2		0
4	P3	test(*s*)	P2	P3	0
5	P4	test(*s*)	P2	P3, P4	0
6	P2	increment(*s*)	P3	P4	0
7			P3	P4	0
8	P3	increment(*s*)	P4		0
9	P4	increment(*s*)			1

Sayma Semaforu (Counting Semaphore)

- Negatif tamsayı değerleri de alabilen semaforlar Sayma Semaforu (Counting Semaphore) ya da genel semaforlar olarak adlandırılır.

Sayma Semaforu (Counting Semaphore)
wait() ve signal() komut eşdeğer yapısı

```
wait(S):
  S.count--;
  if (S.count<0)
    {
     Bu process'i bloke durumuna al
     Bu process'i Semafor kuyruğuna yerleştirin
    }
```

BLOCK

```
signal(S):
  S.count++;
  if (S.count<=0)
   {
    Semafor kuyruğundan bir P process'ini çıkar
    Bu P process'ini hazır (ready) kuyruğuna yerleştir
   }
```

WAKE UP

Sayma Semaforu (Counting Semaphore)

- **Block()**

➢ Process durumunu koşuyor (running) durumundan, bekliyor (waiting) durumuna çevirir.

➢ Process'i semafor bekleme (waiting) kuyruğuna yerleştirir.

- **Wake up()**

➢ Semafor bekleme kuyruğundan semaforla ilişkili olan bir process'i çıkarır ve hazır (ready) kuyruğuna koyar.

❑ Block() ve wakeup(): işletim sistemine ait standart sistem çağrıları ile sağlanır.

Sayma Semaforu (Counting Semaphore)

- Bir process, wait() operasyonunu işletirse ve semaforun değerinin negatif olduğunu belirlerse beklemelidir (busy waiting).

- Meşgul bekleme (busy waiting) yerine process bloke edilir. Bloke edilen process semaforla ilişkilendirilir ve durumu “bekler-(waiting)” konumuna getirilir.

- Bu şekilde bir semafor üzerinde bekleyen process’ler Semafor kuyruğunda tutulur.

Sayma Semaforu (Counting Semaphore)

- Semafor S’nin üzerinde bekleyen, bloke edilmiş olan bir process, diğer process’lerden herhangi birinin **signal()** operasyonu işlettiğinde yeniden başlatılabilir.

- Process bir “wake up” operasyonu ile restart edilir.

- Bu işlem, process durumunu ‘beklerden (waiting)’ ‘hazır (ready)’ durumuna geçirir.

Sayma Semaforu (Counting Semaphore)

- Her semafor, tamsayı (integer) bir değere sahiptir ve bu semafor üzerinde bekleyen process listesine sahiptir (process control block-PCB) listesine bir pointer kaydı).

- Bir process bir semafor üzerinde bekliyor ise semaforun process listesine eklenir.

- signal() operasyonu process'i semafor listesinden çıkarır.

Semafor Çalışma Mantığı

Semaphore value : 1

Semaphore list :

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 1

Semaphore list :

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 1

Semaphore list :

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 1

Semaphore list :

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 0

Semaphore list :

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 0

Semaphore list :

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 0

Semaphore list :

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 0

Semaphore list :

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 0

Semaphore list :

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process Switch

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 0

Semaphore list :

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 0

Semaphore list :

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 0

Semaphore list :

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 0

Semaphore list :

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list :

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list :

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list :

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list : B

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list : B

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list : B

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); **BLOCKED**
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list : B

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

BLOCKED

Process Switch

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list : B

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list : B

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list : B

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list : B

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); **BLOCKED**
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -2

Semaphore list : B

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); **BLOCKED**
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -2

Semaphore list : B

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); **BLOCKED**
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -2

Semaphore list : B

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -2

Semaphore list : C B

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -2

Semaphore list : C B

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -2

Semaphore list : C B

```
Wait()
{value=value-1;
if(value<0)
    { add process to list;
      block process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Process C

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -2

Semaphore list : C B

Process Switch

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Process C

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -2

Semaphore list : C B

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Process C

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -2

Semaphore list : C B

```
Signal()
{value=value + 1;
if(value<=0)
    { remove process from list;
      wakeup process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Process C

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -2

Semaphore list : C B

```
Signal()
{value=value + 1;
if(value<=0)
    { remove process from list;
      wakeup process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Process C

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list : C B

```
Signal()
{value=value + 1;
if(value<=0)
    { remove process from list;
      wakeup process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Process C

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list : C B

```
Signal()
{value=value + 1;
if(value<=0)
    { remove process from list;
      wakeup process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Process C

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list : C B

```
Signal()
{value=value + 1;
if(value<=0)
    { remove process from list;
      wakeup process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Process C

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list : C

```
Signal()
{value=value + 1;
if(value<=0)
    { remove process from list;
      wakeup process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Process C

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list : C

```
Signal()
{value=value + 1;
if(value<=0)
    { remove process from list;
     wakeup process;
     }
 }
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Process C

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list : C

```
Signal()
{value=value + 1;
if(value<=0)
    { remove process from list;
      wakeup process;
    }
}
```

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process Switch

Process B

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Process C

Start
Wait(); BLOCKED
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list : C

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait(); **BLOCKED**
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list : C

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait(); **BLOCKED**
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : -1

Semaphore list : C

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait(); **BLOCKED**
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 0

Semaphore list : C

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait(); **BLOCKED**
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 0

Semaphore list :

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait(); **BLOCKED**
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 0

Semaphore list :

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 0

Semaphore list :

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 0

Semaphore list :

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 1

Semaphore list :

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

Semaphore value : 1

Semaphore list :

Process A

Start
Wait();
Critical section
Signal();
Do remainder

Process B

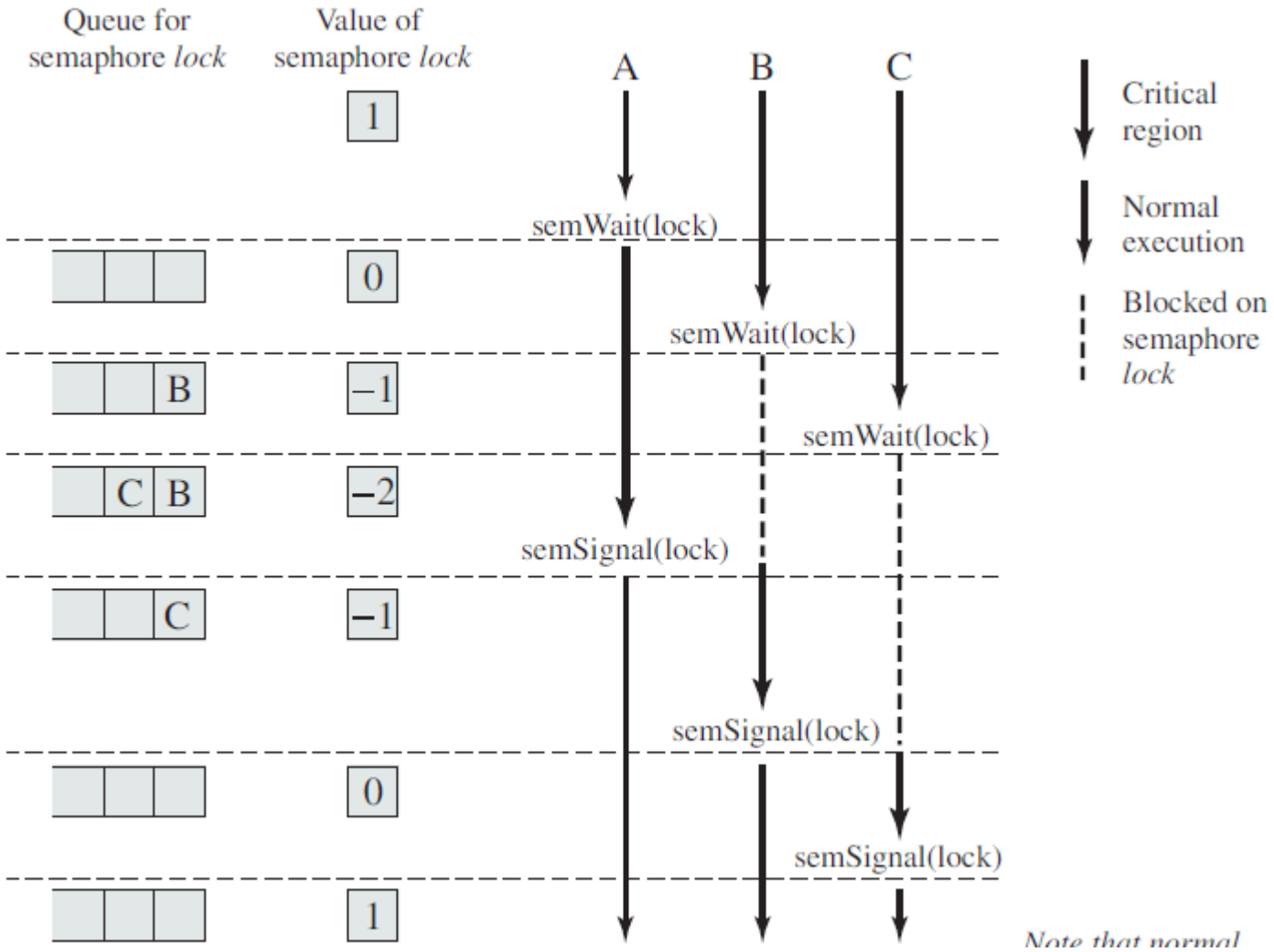
Start
Wait();
Critical section
Signal();
Do remainder

Process C

Start
Wait();
Critical section
Signal();
Do remainder

Semafor Çalışma Mantığı

- Semafor kullanarak kritik bölge işletimini gerçekleştiren üç process.



İkili Semafor ile Sayım Semaforunu Uygulamak

- Data structures:

  binary-semaphore S1, S2;

  int C;
- Initialization:

  S1 = 1

  S2 = 0

  C = initial value of semaphore **S**

- *wait* operation:

```
wait(S1);
C--;
if (C < 0) {
        signal(S1);
        wait(S2);   }
signal(S1);
```

- *signal* operation:

```
wait(S1);
C ++;
if (C <= 0)
      signal(S2);
else signal(S1);
```

Semaforlar (Semaphore)

- Semafor ile işlem yapmak, birlikte çalışan (cooperative) process'lerin yönetimine hem kuramsal hem de uygulanırlık açısından gerekli ve yeterli desteği sağlıyor olmalarına karşın bazı tasarım ve programlama yükünü de birlikte getirirler.
- Process senkronizasyonu, process'lerin içine P() ve V() işlemlerinin serpiştirilmesini gerektirir.
- Bu durum programların okunurluğu ve anlaşılmasını olumsuz yönde etkiler.
- Bunun yanı sıra bu işlemlerin yanlış kullanımı, kimi zaman sistem kaynaklarının kilitlenmesine, dolayısıyla sistemin başarımının düşmesine ve güvenirliğinin zedelenmesine de neden olabilir.

Semaforlar (Semaphore)

- Semaforlar kullanıldığında da senkronizasyon hataları olabilmektedir.
- Tüm process'lerin kritik bölüme girmeden önce wait(), sonra ise signal() işlemlerini yapmaları gereklidir.
- Program geliştirici bu sıraya dikkat etmezse, iki veya daha fazla process zaman paylaşımlı olarak eşzamanlı kritik bölgelerini yürütüyor olabilirler.
- Bu durumlar programcılar arasında yeterli işbirliği olmadığı durumlarda da olabilmektedir.

İzleyiciler (Monitors)

- Monitor’ler, paylaşılan kaynakların yönetimi ve tanımlanması için kullanılan bir soyut veri yapısıdır.
- Monitor kavramı ilk kez C.A.R. Hoare tarafından 1974'te tanımlanmıştır ve Pascal dilinde desteklenmiştir.
- Monitor’ler, senkronizasyonun programlama diline entegre edilmesi esasına dayanır.
- Böylece derleyici, senkronizasyon için kullanımı daha kolay, daha yüksek düzeyde bir arayüz sağlamak üzere işletim sistemiyle birlikte çalışır.
- Monitor’leri, destekleyen bazı diller Java, C# (.NET), Python, Modula-2/3, Visual Basic, Ada, vs.

İzleyiciler (Monitors)

- Monitor, bir nesne veya veri yapısıyla ilişkili olan bir senkronizasyon mekanizmasıdır.
- Monitor, semaforlarla karşılaştırıldığında daha iyi yapılandırılmıştır ve bu nedenle anlaşılması ve uygulanması daha kolaydır.
- Monitor, veri erişimi ve senkronizasyon için üst düzey bir soyutlama sağlayarak process senkronizasyonunu basitleştiren daha yüksek seviyeli bir senkronizasyon yapısıdır.
- Monitor, genellikle nesne yönelimli dillerde programlama dili yapıları olarak uygulanır ve tek bir yapıda karşılıklı dışlama (mutual exclusion), koşul değişkenleri (condition variables) ve veri kapsülleme (data encapsulation) sağlar.

İzleyiciler (Monitors)

- Birlikte çalışan process'ler programlanırken ortak kaynağa erişim yapacak monitör prosedürleri (kritik bölgeler) özel bir kümede toplanır.

- Programlama aşamasında bu küme özel bir tür olarak, monitor biçiminde tanımlandığından, derleme aşamasında bu kümenin başına, karşılıklı dışlamayı sağlayacak gerekli karşılıklı dışlama komutları derleyici tarafından eklenir.

```
monitor monitor_name{

    //declaring shared variables

  variables_declaration;
  condition_variables;

  procedure p1(...) {
   ...
  };

  procedure p2(...){
    ...
  };
  ...
  procedure pn(...){
    ..
  };

 {
   Initailisation Code();
 }
}
```

İzleyiciler (Monitors)

- Monitor, birlikte çalışan process'lerin (cooperative processes), ortak kaynağa erişim yapan yordamlarının (kritik bölgelerinin) toplandığı kümeye verilen addır.
- Bu kümenin tüm process'ler tarafından bilinen özel bir adı bulunur.
- Yürütme aşamasında bir process ortak kaynağa erişmek istediğinde, ilgili yordamı (procedure) bu kümeden çağırır.
- Bu kümeden, aynı anda en çok bir process'in yordam çağırmasına izin verilerek ortak kaynağa aynı anda en çok bir process'in erişim yapması sağlanır.

İzleyiciler (Monitors)

- Bir monitor tarafından sağlanan yordamlar (procedures), herhangi bir zamanda paylaşılan kaynağa (kritik bölge koduna) yalnızca bir process'in erişebilmesini ve kaynağı bekleyen process'lerin, kaynak kullanılabilir hale gelinceye kadar bloke olmasını sağlar.

```
monitor monitor_name{

    //declaring shared variables

  variables_declaration;
  condition_variables;

  procedure p1(...) {
   ...
  };

  procedure p2(...){
    ...
  };
  ...
  procedure pn(...){
    ..
  };

 {
   Initailisation Code();
 }
}
```

İzleyiciler (Monitors)

- Process’lerin monitör’de yer alan bir yordamı (procedure) çağırmaları monitör’e girmeleri olarak nitelenir.
- Bir process monitor içindeki bir yordamın yürütmesini tümüyle tamamlamadan diğer bir process monitor içinden yordam çağıramaz.
- Başka bir deyişle **bir process, monitor içinde iken başka bir process monitore giremez.**
- Bu amaçla, monitör’e ilişkin bir bekleme kuyruğu da öngörülür.

İzleyiciler (Monitors)

- Monitor’ler genellikle **3 temel bileşenden** oluşur:
- **a) Paylaşılan Veri Yapısı (Shared Data)**
- Monitör tarafından korunan veri (örneğin, bir buffer, queue, counter vb.).
- **b) Koşul Değişkenleri (Condition Variables)**
- Thread'lerin belirli koşullar altında bekletilmesini sağlar. (örneğin wait() ve signal() gibi işlemler) bulunur.
- **c) Erişim prosedürleri (Access procedures)**
- Paylaşılan veriye erişen yordamlar (monitör prosedürleri).
- Aynı anda yalnızca bir process veya thread bu yordamları çalıştırabilir.

```
monitor MonitorAdı {

  // a) Paylaşılan veriler
  veri1, veri2, ..., veriN;

  // b) Koşul değişkenleri
  condition kosul1, kosul2;

  // c) Erişim prosedürleri
  procedure P1(...) {
    // kritik bölge
    if (koşul)
      kosul1.wait();      // beklemeye geç
    ...
    kosul2.signal();      // başka process’i uyandır
  }

  procedure P2(...) {
    ...
  }
}
```

İzleyiciler (Monitors)

- Koşul Değişkenleri (Condition Variables)
- **a) wait(condition_variable)**
- Bir process veya thread, belirli bir koşul sağlanana kadar beklemeye geçer.

- **b) signal(condition_variable)**
- Bekleyen bir process veya thread'i uyandırır (sadece bir process veya thread etkilenir).

```
monitor MonitorAdı {

  // Paylaşılan veriler
  veri1, veri2, ..., veriN;

  //  Koşul değişkenleri (Koşul değişkeni tanımı)
  condition kosul1, kosul2;

  // Erişim prosedürleri
  procedure P1(...) {
    // kritik bölge
    if (koşul)
      kosul1.wait();      // beklemeye geç
    ...
    kosul2.signal();      // başka process'i uyandır
  }
  procedure P2(...) {
    ...
  }
}
```

İzleyiciler (Monitors)

- Monitor ile ilgili önemli bir sorun, monitör'e bir kez girebilmiş bir process'in burada, karşılıklı dışlama gereğinin dışında, başka bir nedenle beklemesi gerektiğinde bu beklemenin, diğer process'leri kilitlemeden nasıl gerçekleşeceği sorunudur.

- Bu sakıncayı ortadan kaldırmak için kimi koşullarda, bekleme durumundaki bir process, halen monitor içinde iken, diğer bir process'in de monitör'e girişinin sağlanması gerekir.
- Bunun için, monitör'de, koşul (condition) adlı yeni bir değişken türü ile bunlar üzerinde işlem yapan **wait** ve **signal** adlı komutlar tanımlanır.

İzleyiciler (Monitors)

- Programcı işe veya değişkene özel senkronizasyon oluşturmak için durum değişkenleri oluşturabilir.
- **condition x, y**
- Sadece bir durum değişkenine bağlı çalışan wait() ve signal() işlemleri tanımlanabilir.
- **x.wait()**; ile x durum değişkenine bağlı bir process beklemeye alınır.
- **x.signal()**; ile x durum değişkenine bağlı beklemekte olan process'lerden birine çalışmaya devam etmesi için şans verilir.

İzleyiciler (Monitors)

- Monitor içinde tanımlanan bir yordam (procedure), sadece monitor içinde tanımlanan değişkenlere ve kendi parametrelerine erişebilir.
- Monitor içindeki yordamlardan sadece bir tanesi aynı anda aktif olabilir.
- Bir thread monitöre girdiğinde otomatik olarak kilidi alır.
- Eğer bir koşul sağlanmıyorsa (wait çağrılır), process veya thread beklemeye geçer ve kilidi serbest bırakır.
- signal ile başka bir process veya thread uyandırabilir.
- Process veya thread monitörden çıkarken kilidi serbest bırakır.

```
monitor MonitorAdı {

  // 1. Paylaşılan veriler
  veri1, veri2, ..., veriN;

  // 2. Koşul değişkenleri (Koşul değişkeni tanımı)
  condition kosul1, kosul2;

  // 3. Erişim prosedürleri
  procedure P1(...) {
    // kritik bölge
    if (koşul)
      kosul1.wait();      // beklemeye geç
    ...
    kosul2.signal();      // başka process'i uyandır
  }
  procedure P2(...) {
    ...
  }
}
```

Monitor Kullanımının Avantajları

- Basit ve güvenli bir senkronizasyon yöntemi sunar.
- Otomatik kilitleme özelliğiyle hata yapma olasılığı azalır.
- Daha güvenli kod.
- Koşul değişkenleri sayesinde esnek bekleme/uyandırma mekanizması sağlar.
- Soyutlama sayesinde kullanım kolaylığı

Monitor Kullanımının Sınırlamaları

- Donanımsal düzeyde destek yok.
- Dil desteği gerektirir. Donanım düzeyinde desteklenmediği için genellikle programlama dillerinin (Java, C#, Python vb.) yapılarıyla sağlanır.
- Düşük seviye kontrol sınırlı. Karmaşık senkronizasyon ihtiyaçlarında yeterli esnekliği sağlayamayabilir.
- Performans açısından alt seviye mekanizmalardan (mutex, semaphore) daha yavaş olabilir.

Monitor için Örnek Senaryo

- Bu senaryoda, **count** adında paylaşılan bir değişkeni kullanan iki process var: P1 (count'u bir artırır) ve P2 (count'u bir eksiltir).
- Bu process'lerin eşzamanlı yürütülmesi durumunda yarış koşulu (race condition) oluşabilir.
- Monitor kullanarak bu process'ler senkronize edilebilir.
- Paylaşılan Değişken: **count** (başlangıç değeri, örneğin 0).
- P1: **count = count + 1** (artırma işlemi).
- P2: **count = count - 1** (eksiltme işlemi).
- Amaç: Yarış koşulunu önlemek ve **count** değişkeninin tutarlılığını sağlamak.
- Monitor, karşılıklı dışlama (mutual exclusion) sağlayarak aynı anda yalnızca bir process'in count üzerinde işlem yapmasına izin verecek.

Monitor için Örnek Senaryo

- **Monitor Tanımlaması**

```
monitor Counter {
   int count = 0; // Paylaşılan değişken

   procedure increment() {
      count = count + 1; // P1'in yapacağı işlem
   }

   procedure decrement() {
      count = count - 1; // P2'nin yapacağı işlem
   }
}
```

- increment: P1’in çalıştıracağı metot.
- decrement: P2’nin çalıştıracağı metot.

- Koşul değişkenine bu basit senaryoda gerek yok, çünkü yalnızca karşılıklı dışlama yeterli.
- Ancak daha karmaşık bir senaryo (örneğin, count belirli bir değerde beklemeli) için koşul değişkeni eklenebilir.

Monitor için Örnek Senaryo

- Adım 1: P1 Monitor’e Girer

- P1, increment metodunu çağırır.
- Monitor kilidi devreye girer:
  - P1, monitor’e girer ve kilidi alır.
  - P2 (veya başka bir process) bu sırada monitor’e giremez, bekler.

- Durum: count = 0, kilit P1’de.

Monitor için Örnek Senaryo

- Adım 2: P1 İşlemi Gerçekleştirir

- P1, count = count + 1 işlemini yapar:
- Başlangıç: count = 0

- Sonuç: count = 1

- Bu işlem atomik olarak gerçekleşir, çünkü monitor P2’nin aynı anda müdahale etmesini engeller.

- Durum: count = 1, kilit hâlâ P1’de.

Monitor için Örnek Senaryo

- Adım 3: P1 Monitor’den Çıkar
- P1, increment metodunu tamamlar ve monitor’den çıkar.

- Monitor kilidi serbest bırakılır.

- Durum: count = 1, kilit serbest.

Monitor için Örnek Senaryo

- Adım 4: P2 Monitor’e Girer
- P2, decrement metodunu çağırır.

- Monitor kilidi serbest olduğu için P2 kilidi alır ve monitor’e girer.

- P1 bu sırada tekrar girmek isterse bekler.

- Durum: count = 1, kilit P2’de.

Monitor için Örnek Senaryo

- Adım 5: P2 İşlemi Gerçekleştirir
- P2, count = count - 1 işlemini yapar:
- Başlangıç: count = 1.

- Sonuç: count = 0.

- Bu işlem de atomik olarak gerçekleşir.

- Durum: count = 0, kilit hâlâ P2’de.

Monitor için Örnek Senaryo

- Adım 6: P2 Monitor’den Çıkar
- P2, decrement metodunu tamamlar ve monitor’den çıkar.

- Monitor kilidi serbest bırakılır.

- Durum: count = 0, kilit serbest.

- Adım 7: Süreç Tekrarlanır
- P1 ve P2, ihtiyaç duydukça monitor’e sırayla girip işlemlerini yapar.
- Monitor, her zaman yalnızca bir process’in çalışmasını sağlar.

Monitor için Örnek Senaryo

- **Koşul Değişkeni ile Genişletme**

- Eğer count'un belirli bir değerde beklemesi gerekirse (örneğin, P2 count = 0 iken eksiltme yapamasın), monitor'e koşul değişkeni eklenir:

```
monitor Counter {
   int count = 0;
   condition notZero; // count sıfır değilse

   procedure increment() {
      count = count + 1;
      signal(notZero); // P2'yi uyandır
   }

   procedure decrement() {
      if (count == 0) {
         wait(notZero); // count sıfırsa bekle
      }
      count = count - 1;
   }
}
```

Monitor için Örnek Senaryo

- P2 Monitor'e Girer, Bekler:
- count = 0 ise wait(notZero) ile bekler, kilit serbest kalır.

- P1 Monitor’e Girer:
- increment ile count = 1 yapar, signal(notZero) ile P2’yi uyandırır.

- P2 Monitor’deDevam Eder:
- Uyanır, kilidi alır, count = 0 yapar.

Klasik Senkronizasyon Problemleri

- Üretici Tüketici Problemi (Producer Consumer problem)
- Okuyucular – Yazıcılar Problemi (Readers-writers problem)
- Filozofların Akşam Yemeği Problemi (Dining philosopher)
- Uyuyan Berber Problemi (Sleeping Barber)

Üretici - Tüketici Problemi
(Producer - Consumer Problem)

- Aynı anda çalışan iki process bulunmaktadır ve bunlardan birisi üretmekte diğeri de bu üretilen ürünü tüketmektedir.
- Buradaki sorun, üretilen ürün henüz tüketilmeden ikincisinin üretilmemesi veya henüz ürün üretilmeden tüketilmemesidir.
- Problem tanımı itibariyle, process'ler arası senkronizasyonu gerektirmektedir ve iki process birbiri ile senkronize olmalıdır.
- Yani tüketici, ne zaman tüketeceğine, üreticinin hareketlerine bakarak karar vermeli, benzer şekilde üretici de ne zaman üreteceğine, tüketicinin hareketlerine bakarak karar vermelidir.

Üretici - Tüketici Problemi
(Producer - Consumer Problem)

- Veri üreten bir kullanıcı process'i ile bu verileri yazıcıdan döken bir sistem process'i birlikte çalışan iki process olarak ele alındığında;
- Kullanıcı process'inin ürettiği verilerin yazıcı sistem process'i tarafından dökülebilmesi, ilgili verilerin bir process'den diğer process'e aktarılmasını gerektirir.
- Bu aktarım her iki process'in ortak erişebildiği bir tampon bellek (buffer memory) alanı üzerinden gerçekleşebilir.

Üretici - Tüketici Problemi
(Producer - Consumer Problem)

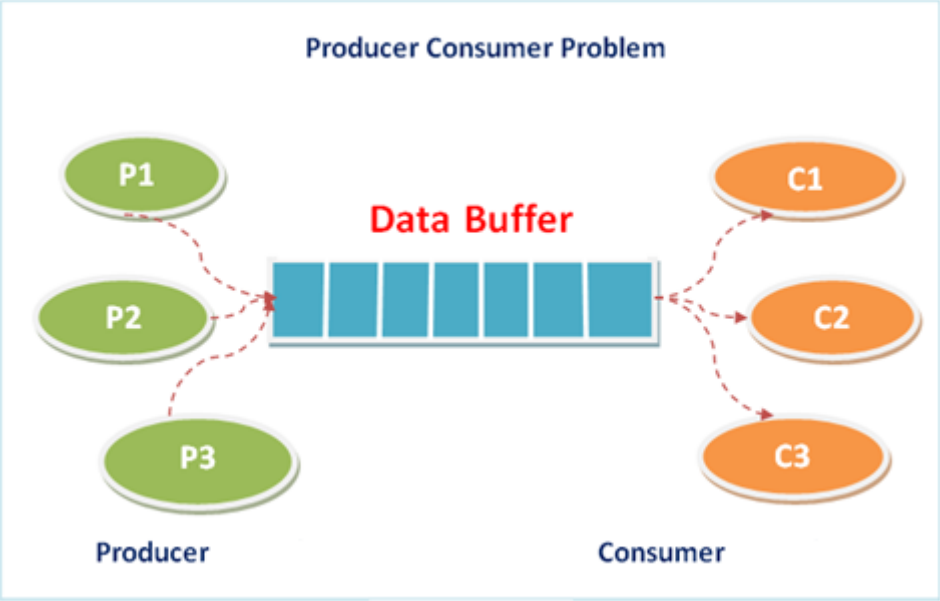
- Kullanıcı process'i, dökümü alınacak veriyi, sırayla bu buffer alanına yazar.
- Yazıcı sistem process'i de, buffer alanından okuduğu verilerin yazıcıdan dökülmesini sağlar.
- Ortak buffer alanına yazma yapan process üretici **(Producer)**,
- Ortak buffer alanından okuma yapan process ise tüketici **(Consumer)** olarak nitelenir.
- Bu nedenle, kullanıcı process üretici, yazıcı sistem process'i ise tüketici process konumundadır.

Üretici - Tüketici Problemi
(Producer - Consumer Problem)

- Ortak bir buffer üzerinden veri alış-verişinde bulunan üretici ve tüketici process'ler birbirlerinin işletimini etkileyen process'lerdir.
- Bu bağlamda, daha önce verilen örnekte yazıcı sistem process'inin işletilebilmesi, kullanıcı process'inin yazdırılacak bir veriyi ortak buffer alanı üzerinde hazır etmesine bağlıdır.
- Bunun gibi kullanıcı process'inin, yeni bir veriyi buffer alanına yazabilmesi, bir önce yazılan verinin yazıcı process tarafından okunup dökülmüş olmasını gerektirir.

Üretici - Tüketici Problemi (Producer - Consumer Problem)

- Birden fazla üretici ve tüketici olması durumunda;
- Örneğin aynı buffer üzerine 3 ayrı process ürettikleri verileri koyabilmekteler veya 3 ayrı process üretilen bu ürünleri tüketebilmektedirler.
- Bu durumda, aynı anda iki üreticinin üretim yapmaması veya aynı anda iki tüketicinin veriyi tüketmemesi gerekir.



Üretici - Tüketici Problemi (Producer - Consumer Problem)

- **Unbounded (sınırlanmamış) buffer** üretici / tüketici probleminde buffer uzunluğunda bir sınır yoktur.

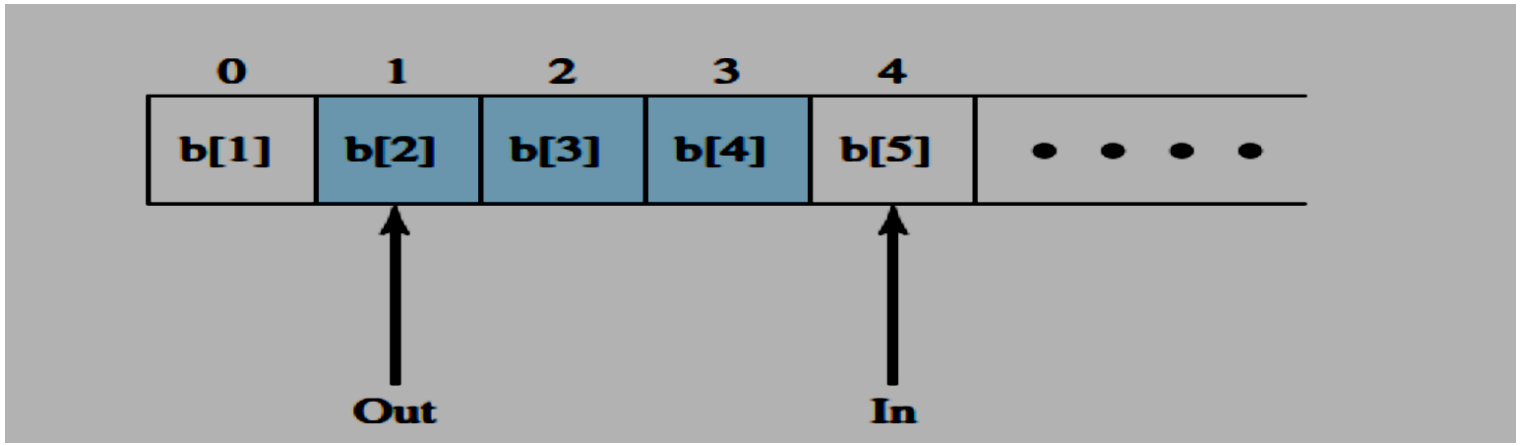
- Tüketici yeni ürünler bekler, ve gelenleri tüketir, sadece ürün bitince bekler fakat üretici daima yeni ürün üretip koyabilir.

Sınırlanmamış buffer

```
producer:
while (true) {
    /* produce item v */;
    b[in] = v;
    in++;
}
```

```
consumer:
while (true) {
    while (in <= out)
        /* do nothing */;
    w = b[out];
    out++;
    /* consume item w */;
}
```

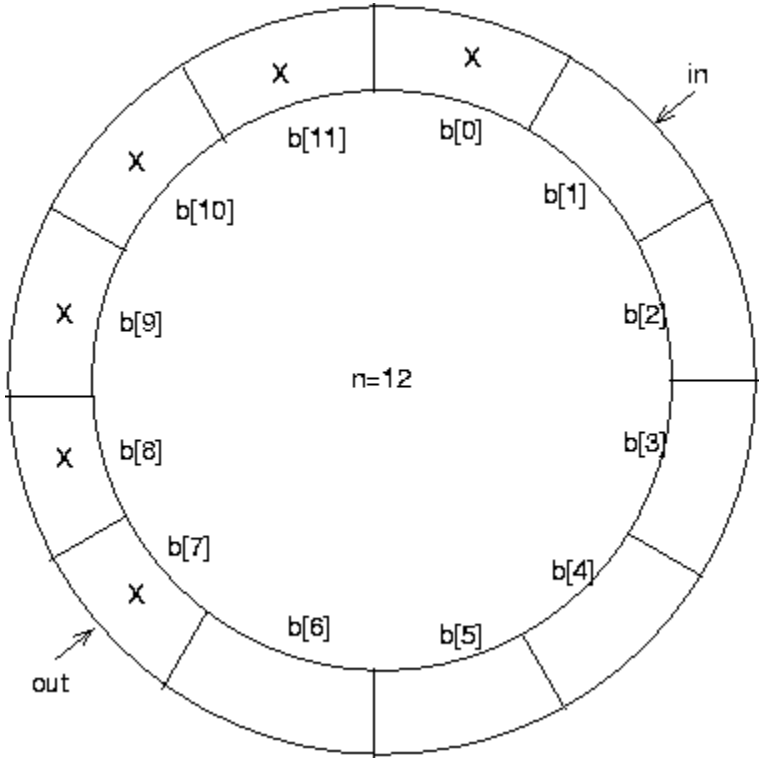
Infinite buffer



Üretici - Tüketici Problemi
(Producer - Consumer Problem)

- **Bounded (sınırlanmış) buffer** üretici / tüketici probleminde buffer uzunluğu sınırlıdır.
- Üretici n buffer uzunluğu dolunca bekler , tüketici n buffer uzunluğu boşalınca bekler.
- Üretici üretmiş olduğu veriyi buffer'da boş yer varsa koyabilir. Yer yok ise tüketicinin buffer'dan bir veriyi almasını beklemek durumundadır.
- Tüketici ise ancak buffer boş değilse veri tüketebilir, aksi taktirde üreticinin buffer'a veri girmesini beklemesi gerekir.

Sınırlı buffer

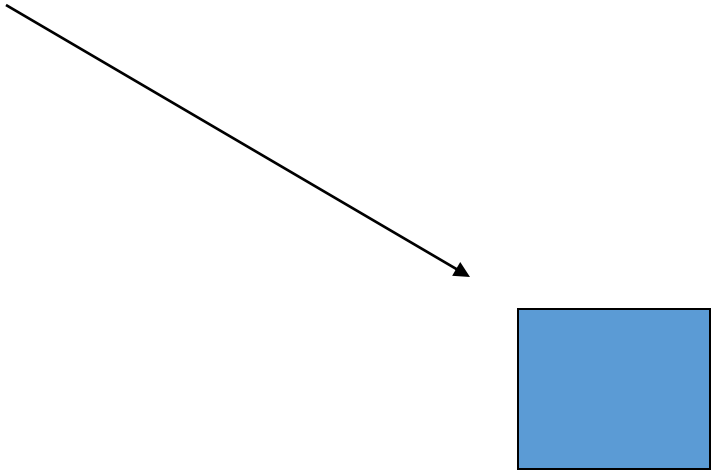



```
producer:
while (true) {
  /* produce item v */
  while ((in + 1) % n == out)
      /* do nothing */;
  b[in] = v;
  in = (in + 1) % n;
}
```

```
consumer:
while (true) {
  while (in == out)
      /* do nothing */;
  w = b[out];
  out = (out + 1) % n;
  /* consume item w */;
}
```

Üretici - Tüketici Problemi (Producer - Consumer Problem)

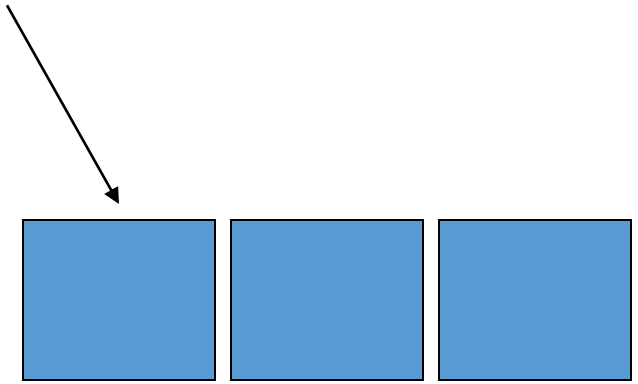
Bir aşçının yemek pişirdiğini ve
onları bir konveyör bant üzerine koyduğunu düşünün



Üretici - Tüketici Problemi (Producer - Consumer Problem)

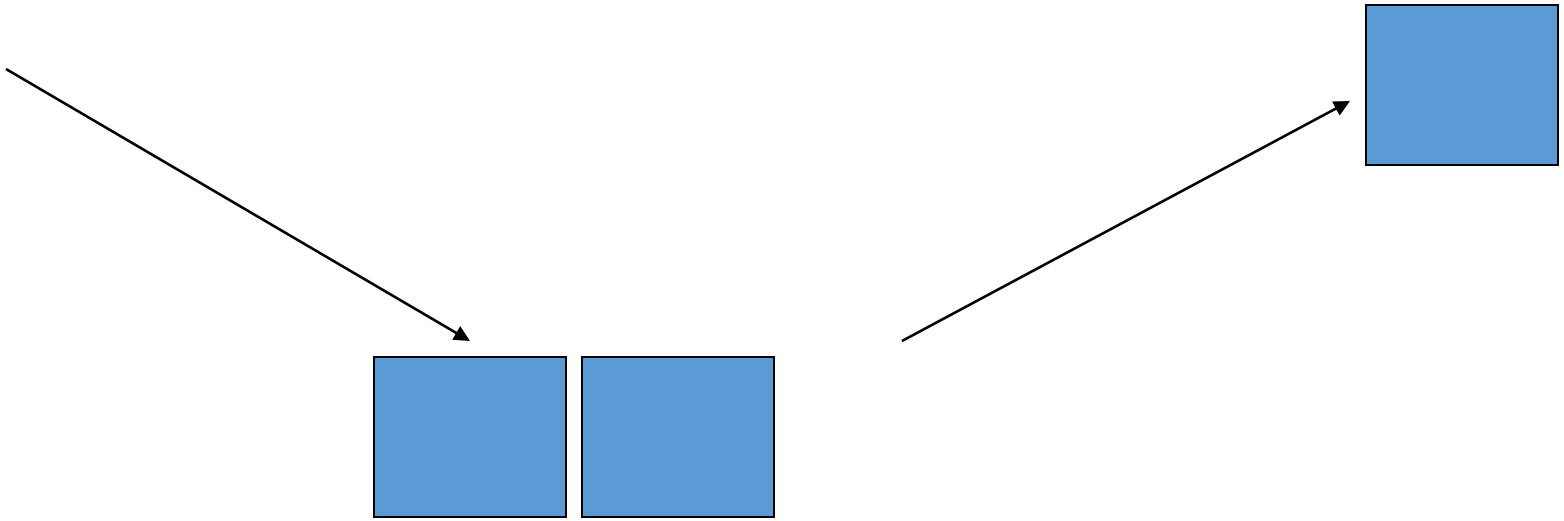
Şimdi birçok aşçıyı hayal edin!

Ve hepsinin yemek pişirdiğini ve
onları bir konveyör bant üzerine koyduğunu düşünün.



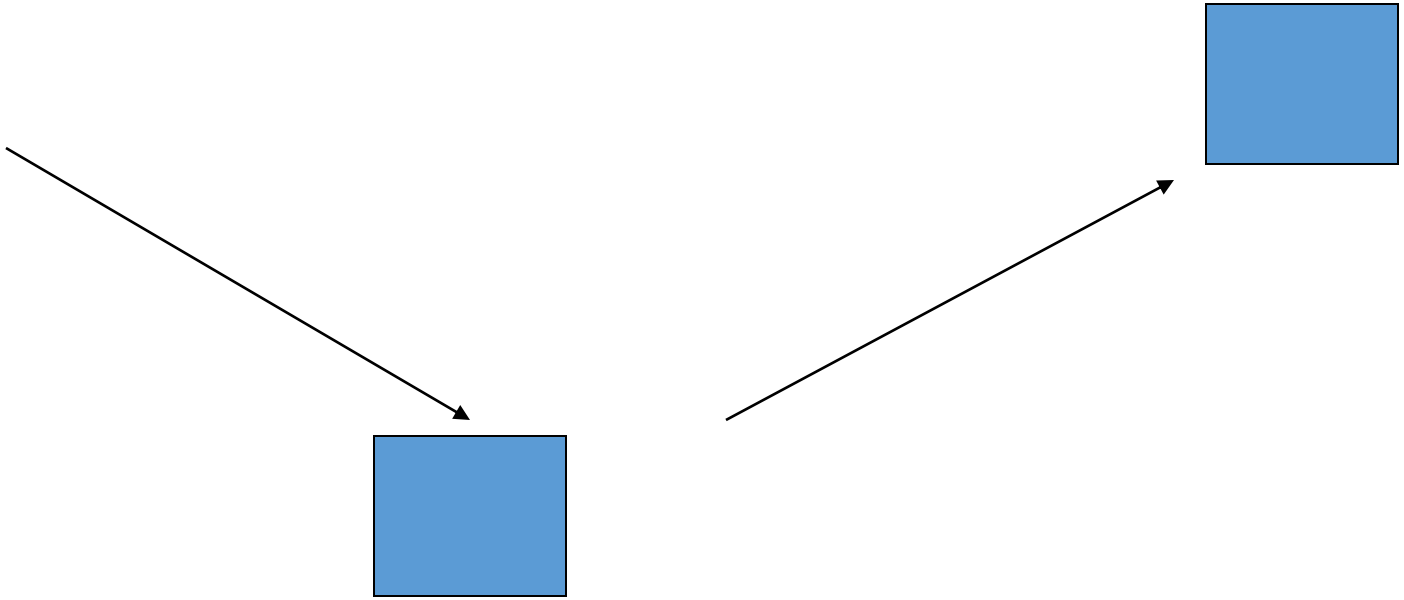
Üretici - Tüketici Problemi (Producer - Consumer Problem)

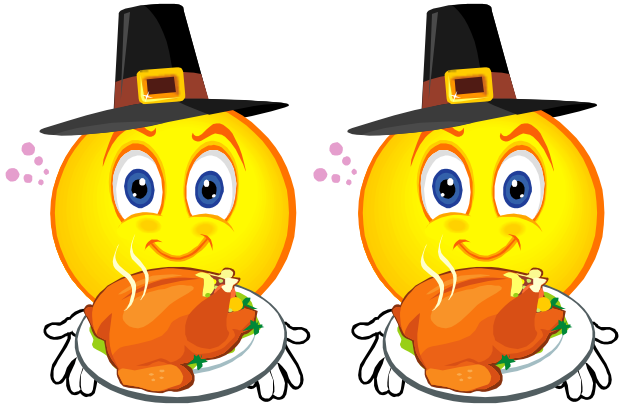
Şimdi bir müşterinin konveyör bandından yemek aldığını hayal edin



Üretici - Tüketici Problemi (Producer - Consumer Problem)

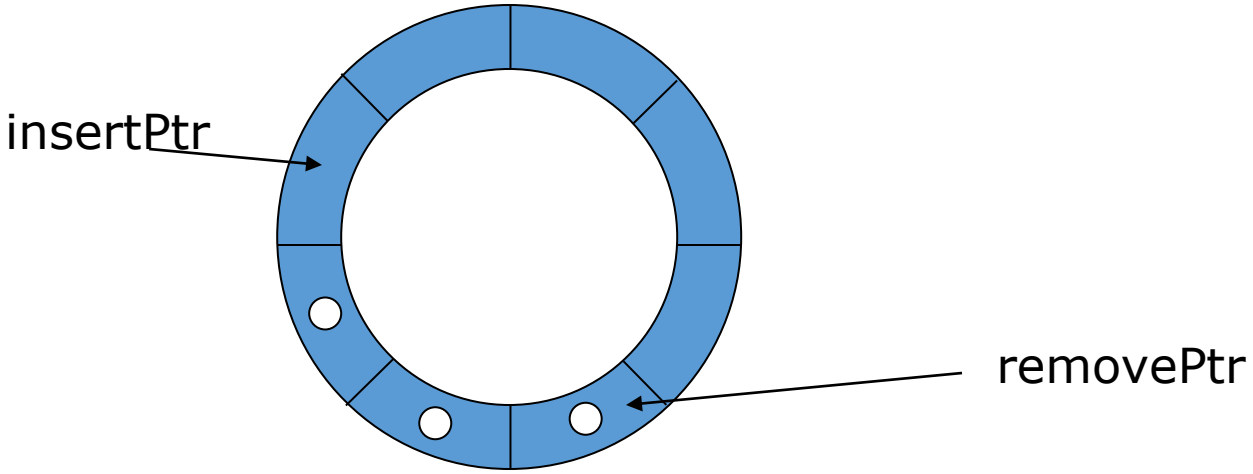
Şimdi birçok müşterinin konveyör bandından yemek aldığını hayal edin!





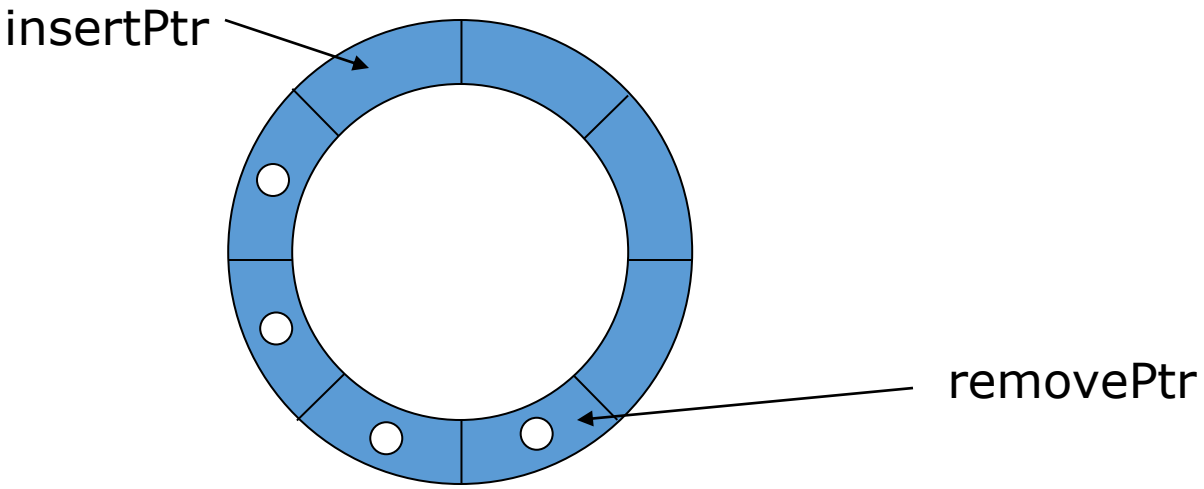
Üretici - Tüketici Problemi (Producer - Consumer Problem)

Aşçı = Üretici (Producer)
Müşteri = Tüketici (Consumer)



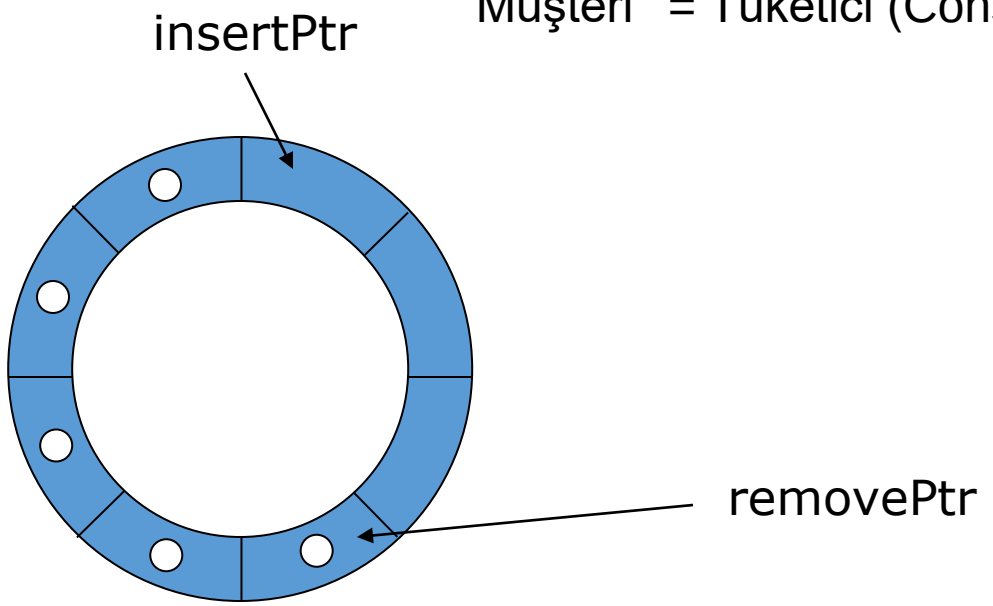
Üretici - Tüketici Problemi (Producer - Consumer Problem)

Aşçı = Üretici (Producer)
Müşteri = Tüketici (Consumer)



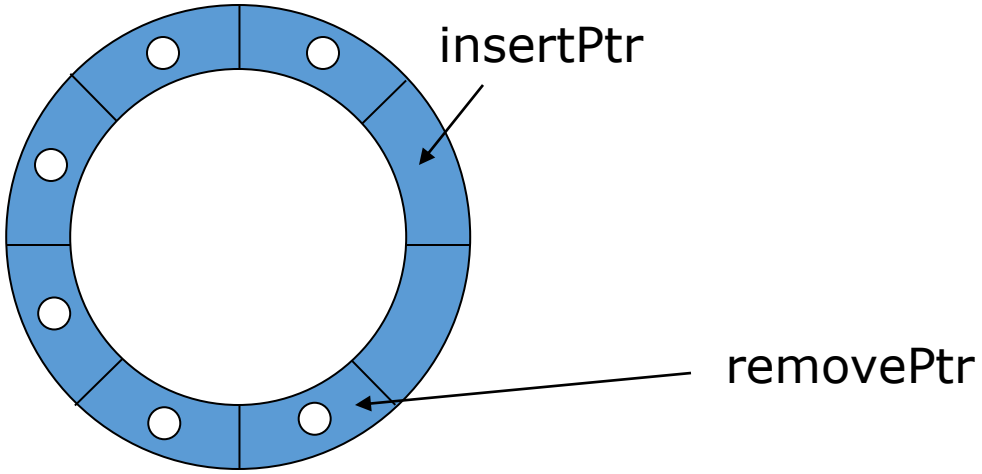
Üretici - Tüketici Problemi (Producer - Consumer Problem)

Aşçı = Üretici (Producer)
Müşteri = Tüketici (Consumer)



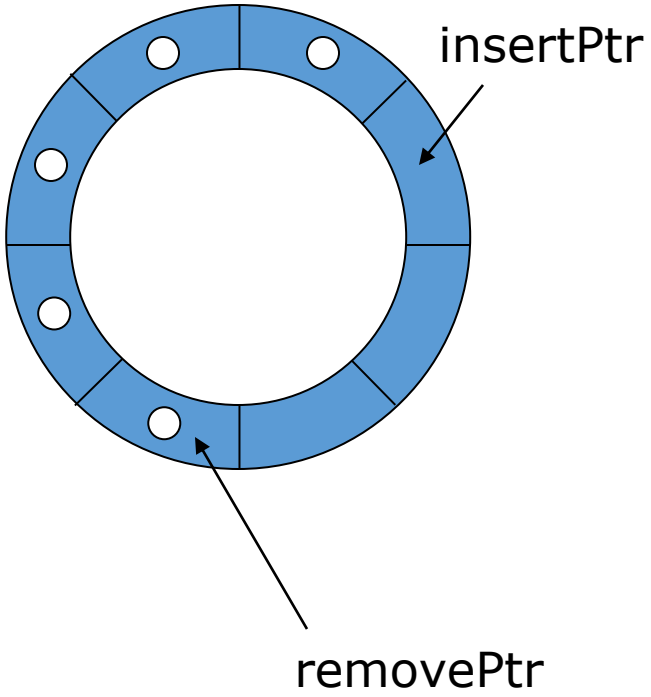
Üretici - Tüketici Problemi (Producer - Consumer Problem)

Aşçı = Üretici (Producer)
Müşteri = Tüketici (Consumer)



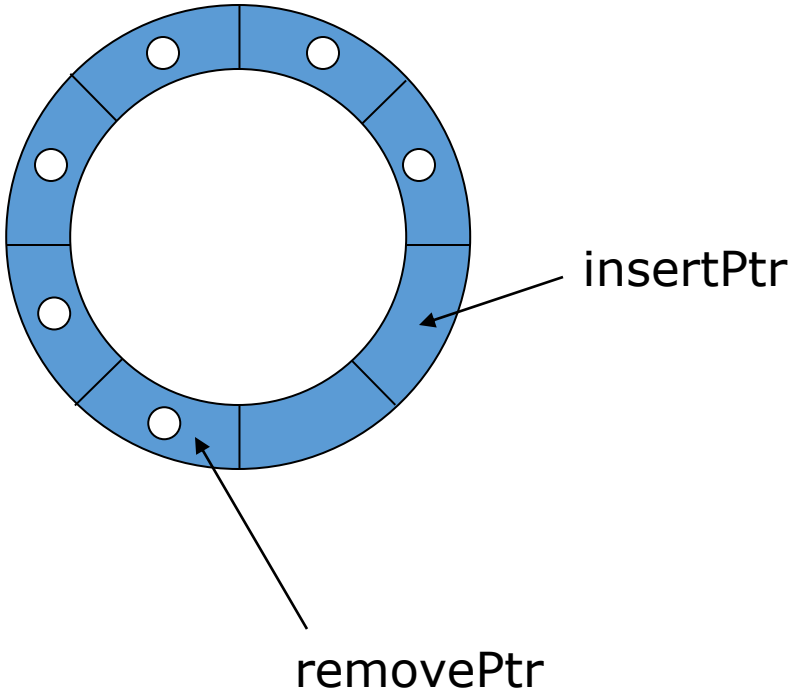
Üretici - Tüketici Problemi (Producer - Consumer Problem)

Aşçı = Üretici (Producer)
Müşteri = Tüketici (Consumer)



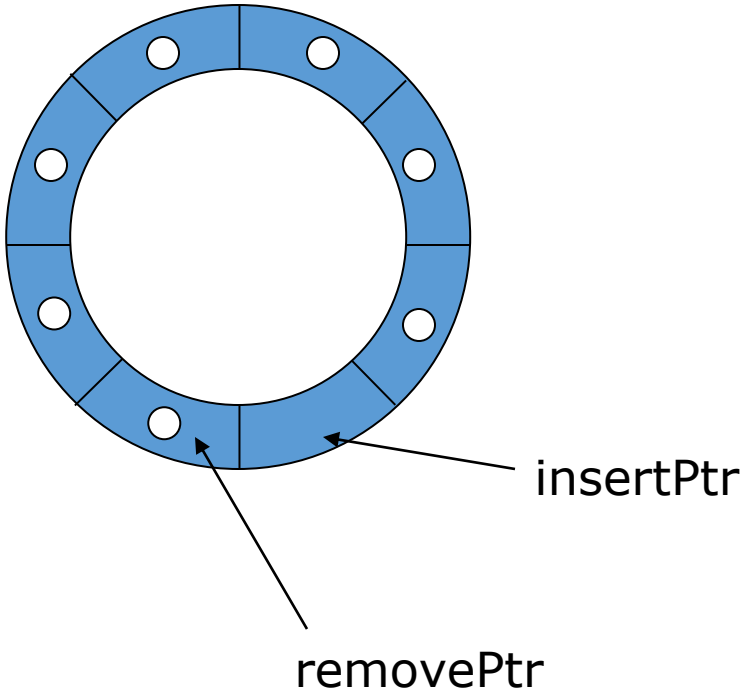
Üretici - Tüketici Problemi (Producer - Consumer Problem)

Aşçı = Üretici (Producer)
Müşteri = Tüketici (Consumer)



Üretici - Tüketici Problemi (Producer - Consumer Problem)

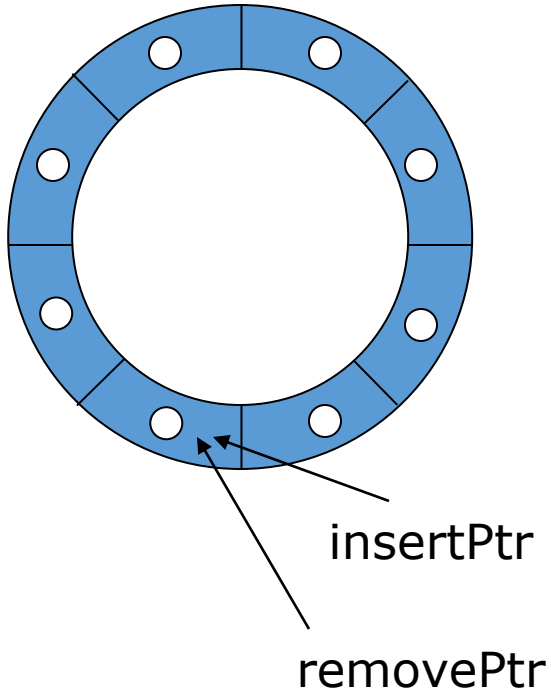
Aşçı = Üretici (Producer)
Müşteri = Tüketici (Consumer)



Üretici - Tüketici Problemi (Producer - Consumer Problem)

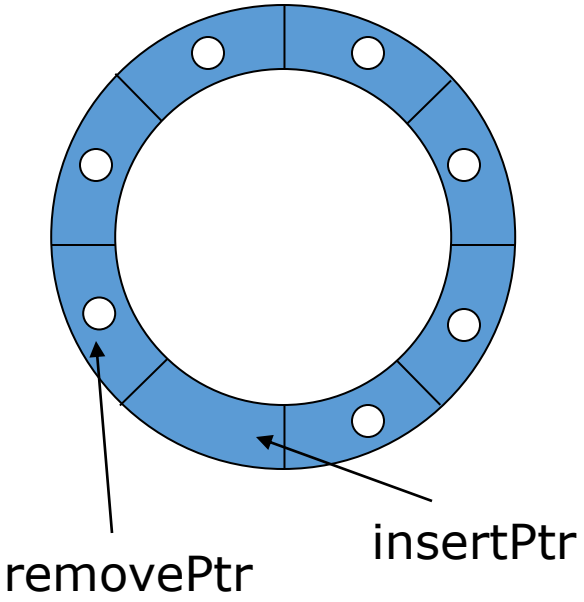
Aşçı = Üretici (Producer)
Müşteri = Tüketici (Consumer)

BUFFER FULL: Producer must be blocked!



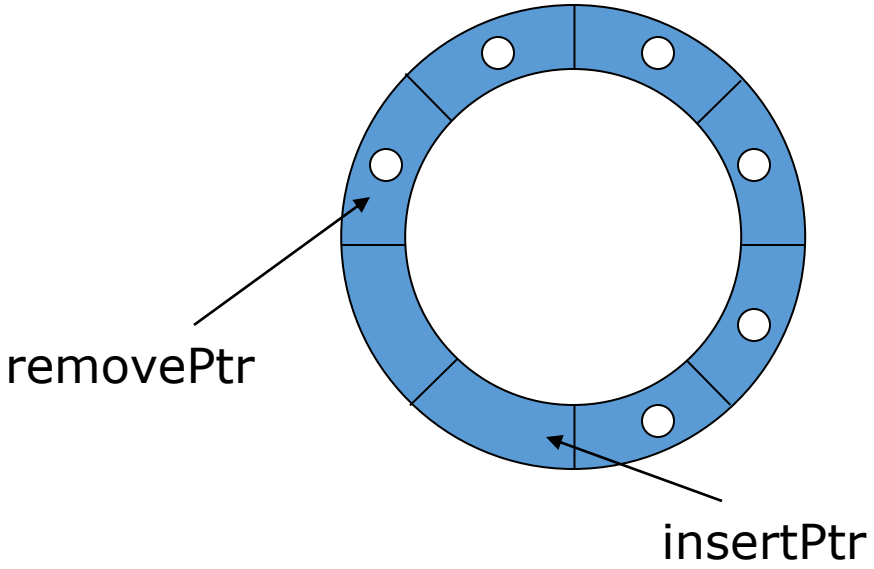
Üretici - Tüketici Problemi (Producer - Consumer Problem)

Aşçı = Üretici (Producer)
Müşteri = Tüketici (Consumer)



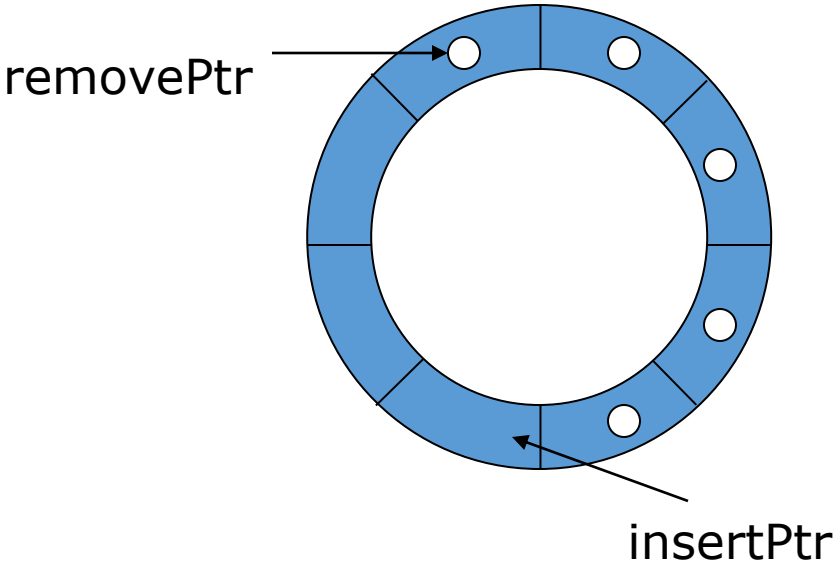
Üretici - Tüketici Problemi (Producer - Consumer Problem)

Aşçı = Üretici (Producer)
Müşteri = Tüketici (Consumer)



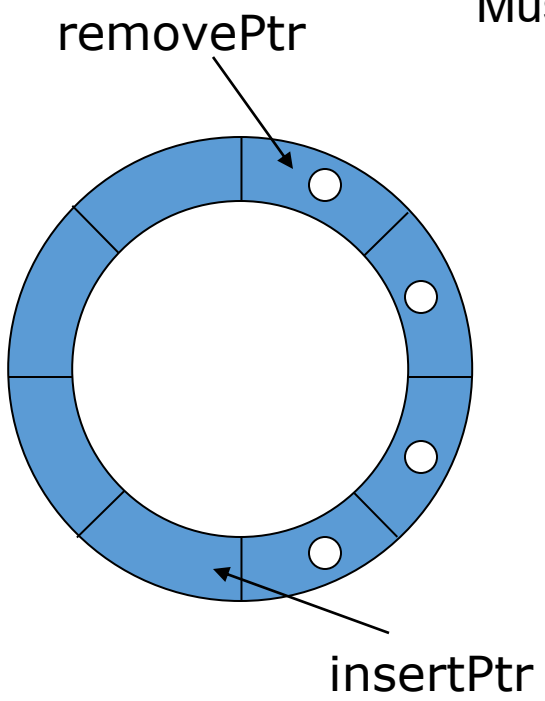
Üretici - Tüketici Problemi (Producer - Consumer Problem)

Aşçı = Üretici (Producer)
Müşteri = Tüketici (Consumer)



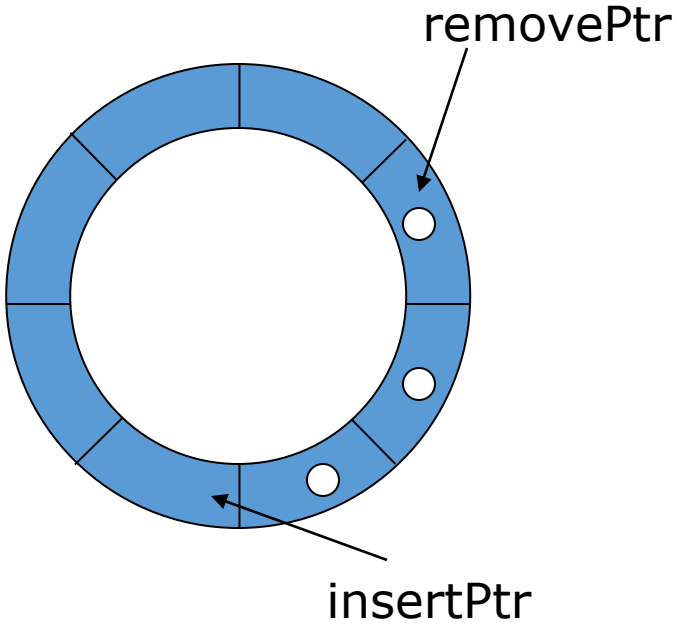
Üretici - Tüketici Problemi (Producer - Consumer Problem)

Aşçı = Üretici (Producer)
Müşteri = Tüketici (Consumer)



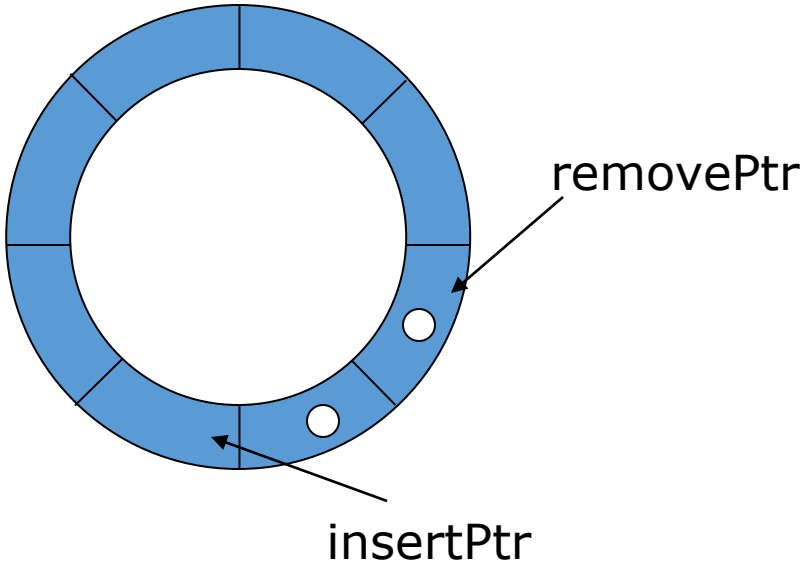
Üretici - Tüketici Problemi (Producer - Consumer Problem)

Aşçı = Üretici (Producer)
Müşteri = Tüketici (Consumer)



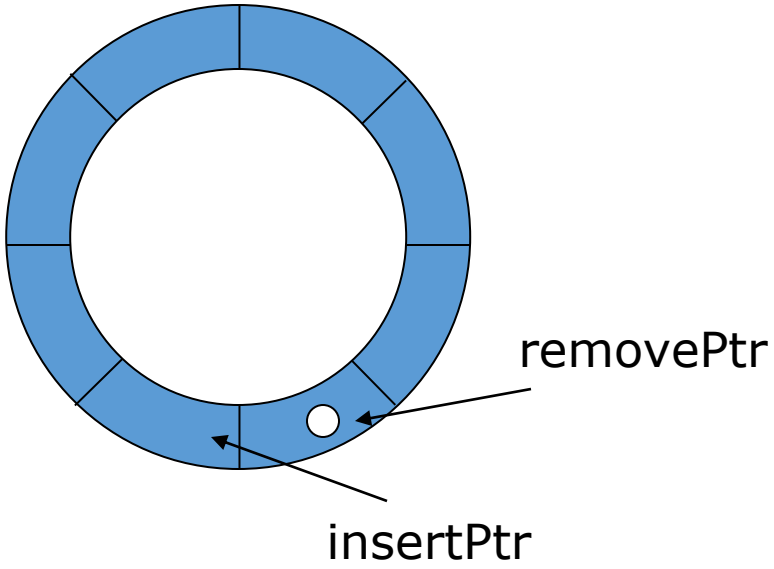
Üretici - Tüketici Problemi (Producer - Consumer Problem)

Aşçı = Üretici (Producer)
Müşteri = Tüketici (Consumer)



Üretici - Tüketici Problemi (Producer - Consumer Problem)

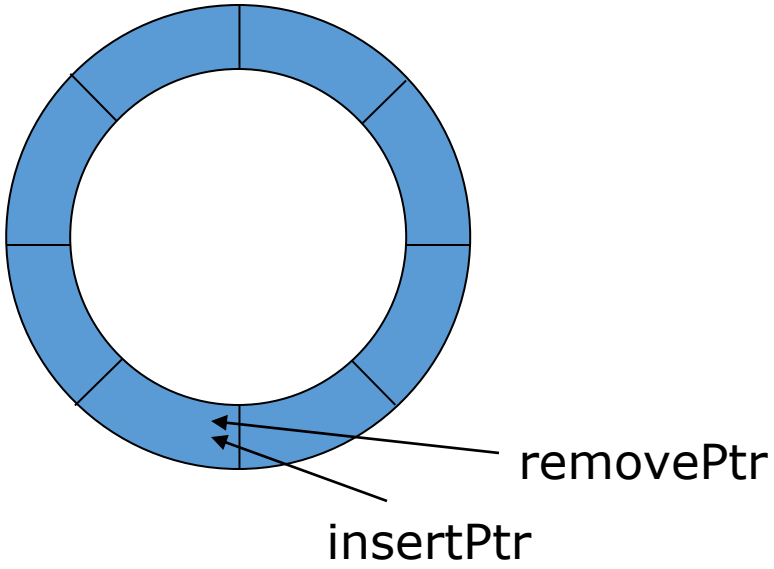
Aşçı = Üretici (Producer)
Müşteri = Tüketici (Consumer)



Üretici - Tüketici Problemi (Producer - Consumer Problem)

Aşçı = Üretici (Producer)
Müşteri = Tüketici (Consumer)

BUFFER EMPTY: Consumer must be blocked!



Üretici-Tüketici Probleminde Process Senkronizasyon Gerekliliği

Producer

```
while (1) {
        /* produce an item in nextProduced */
        while (counter == BUFFER_SIZE)
            ;  /* do nothing */
        buffer[in] = nextProduced;
        in = (in+1) % BUFFER_SIZE);
        counter++;
}
```

Consumer

```
while (1) {
        while (counter == 0)
            ;  /* do nothing */
        nextConsumed = buffer[out];
        out = (out+1) % BUFFER_SIZE);
        counter--;
        /* consume the item in nextConsumed */
}
```

- üretici ve tüketici rutinleri ayrı ayrı doğru olmasına rağmen, uygulandıklarında doğru şekilde çalışmayabilirler.

Üretici-Tüketici Probleminde Process Senkronizasyon Gerekliliği

Producer

```
while (1) {
        /* produce an item in nextProduced */
        while (counter == BUFFER_SIZE)
            ;  /* do nothing */
        buffer[in] = nextProduced;
        in = (in+1) % BUFFER_SIZE);
        counter++;
}
```

Consumer

```
while (1) {
        while (counter == 0)
            ;  /* do nothing */
        nextConsumed = buffer[out];
        out = (out+1) % BUFFER_SIZE);
        counter--;
        /* consume the item in nextConsumed */
}
```

- Counter değişken değerinin şu anda 5 olduğunu ve üretici ve tüketici işleminin ifadelerinin counter++ ve counter-- eşzamanlı olarak çalıştırıldığını varsayalım.
- üreticinin ve tüketicinin çalıştırılmasından sonra doğru sonuç, sayaç = 5 olmalıdır.

Üretici-Tüketici Probleminde Process Senkronizasyon Gerekliliği

Producer

```
while (1) {
        /* produce an item in nextProduced */
        while (counter == BUFFER_SIZE)
                ;  /* do nothing */
        buffer[in] = nextProduced;
        in = (in+1) % BUFFER_SIZE);
        counter++;
}
```

Consumer

```
while (1) {
        while (counter == 0)
                ;  /* do nothing */
        nextConsumed = buffer[out];
        out = (out+1) % BUFFER_SIZE);
        counter--;
        /* consume the item in nextConsumed */
}
```

Counter++

$register_1$ = counter
$register_1$ = $register_1$ + 1
counter = $register_1$

Counter--

$register_2$ = counter
$register_2$ = $register_2$ - 1
counter = $register_2$

T_0: *producer* $register_1$ = counter

5	counter
5	$register_1$
	$register_2$

Üretici-Tüketici Probleminde Process Senkronizasyon Gerekliliği

Producer

```
while (1) {
        /* produce an item in nextProduced */
        while (counter == BUFFER_SIZE)
            ;  /* do nothing */
        buffer[in] = nextProduced;
        in = (in+1) % BUFFER_SIZE);
        counter++;
}
```

Consumer

```
while (1) {
        while (counter == 0)
            ;  /* do nothing */
        nextConsumed = buffer[out];
        out = (out+1) % BUFFER_SIZE);
        counter--;
        /* consume the item in nextConsumed */
}
```

Counter++

$register_1 = counter$
$register_1 = register_1 + 1$
$counter = register_1$

Counter--

$register_2 = counter$
$register_2 = register_2 - 1$
$counter = register_2$

T_0: *producer* $register_1 = counter$
T_1: *producer* $register_1 = register_1 + 1$

5	counter
6	$register_1$
	$register_2$

Üretici-Tüketici Probleminde Process Senkronizasyon Gerekliliği

Producer

```
while (1) {
        /* produce an item in nextProduced */
        while (counter == BUFFER_SIZE)
            ;  /* do nothing */
        buffer[in] = nextProduced;
        in = (in+1) % BUFFER_SIZE);
        counter++;
}
```

Consumer

```
while (1) {
        while (counter == 0)
            ;  /* do nothing */
        nextConsumed = buffer[out];
        out = (out+1) % BUFFER_SIZE);
        counter--;
        /* consume the item in nextConsumed */
}
```

Counter++

$register_1 = counter$
$register_1 = register_1 + 1$
$counter = register_1$

Counter--

$register_2 = counter$
$register_2 = register_2 - 1$
$counter = register_2$

T_0: *producer* $register_1 = counter$
T_1: *producer* $register_1 = register_1 + 1$
T_2: *consumer* $register_2 = counter$

5	counter
6	$register_1$
5	$register_2$

Üretici-Tüketici Probleminde Process Senkronizasyon Gerekliliği

Producer

```
while (1) {
        /* produce an item in nextProduced */
        while (counter == BUFFER_SIZE)
            ;  /* do nothing */
        buffer[in] = nextProduced;
        in = (in+1) % BUFFER_SIZE);
        counter++;
}
```

Consumer

```
while (1) {
        while (counter == 0)
            ;  /* do nothing */
        nextConsumed = buffer[out];
        out = (out+1) % BUFFER_SIZE);
        counter--;
        /* consume the item in nextConsumed */
}
```

Counter++

$register_1 = counter$
$register_1 = register_1 + 1$
$counter = register_1$

Counter--

$register_2 = counter$
$register_2 = register_2 - 1$
$counter = register_2$

T_0: *producer* $register_1 = counter$
T_1: *producer* $register_1 = register_1 + 1$
T_2: *consumer* $register_2 = counter$
T_3: *consumer* $register_2 = register_2 - 1$

5
counter

6
$register_1$

4
$register_2$

Üretici-Tüketici Probleminde Process Senkronizasyon Gerekliliği

Producer

```
while (1) {
        /* produce an item in nextProduced */
        while (counter == BUFFER_SIZE)
            ;  /* do nothing */
        buffer[in] = nextProduced;
        in = (in+1) % BUFFER_SIZE);
        counter++;
}
```

Consumer

```
while (1) {
        while (counter == 0)
            ;  /* do nothing */
        nextConsumed = buffer[out];
        out = (out+1) % BUFFER_SIZE);
        counter--;
        /* consume the item in nextConsumed */
}
```

Counter++

$register_1 = counter$
$register_1 = register_1 + 1$
$counter = register_1$

Counter--

$register_2 = counter$
$register_2 = register_2 - 1$
$counter = register_2$

T_0: *producer* $register_1 = counter$
T_1: *producer* $register_1 = register_1 + 1$
T_2: *consumer* $register_2 = counter$
T_3: *consumer* $register_2 = register_2 - 1$
T_4: *producer* $counter = register_1$

6	counter
6	$register_1$
4	$register_2$

Üretici-Tüketici Probleminde Process Senkronizasyon Gerekliliği

Producer

```
while (1) {
        /* produce an item in nextProduced */
        while (counter == BUFFER_SIZE)
            ;  /* do nothing */
        buffer[in] = nextProduced;
        in = (in+1) % BUFFER_SIZE);
        counter++;
}
```

Consumer

```
while (1) {
        while (counter == 0)
            ;  /* do nothing */
        nextConsumed = buffer[out];
        out = (out+1) % BUFFER_SIZE);
        counter--;
        /* consume the item in nextConsumed */
}
```

Counter++

$register_1 = counter$
$register_1 = register_1 + 1$
$counter = register_1$

Counter--

$register_2 = counter$
$register_2 = register_2 - 1$
$counter = register_2$

T_0: *producer* $register_1 = counter$
T_1: *producer* $register_1 = register_1 + 1$
T_2: *consumer* $register_2 = counter$
T_3: *consumer* $register_2 = register_2 - 1$
T_4: *producer* $counter = register_1$
T_5: *consumer* $counter = register_2$

4
counter

6
$register_1$

4
$register_2$

‘counter = 4' olarak yanlış değer elde ettiğimize dikkat edin. Bu, 5 full buffer olduğu halde, sayacın 4 full buffer gösterdiği anlamına gelir. Her iki process’in eşzamanlı olarak ‘counter' değişkenine geçmesine izin verdiğimiz için bu yanlış duruma varıyoruz.

Semafor ile Üretici-Tüketici Problemi Çözümü

- Üretici-Tüketici probleminde, semaforlar iki amaç için kullanılır:

- karşılıklı dışlanma (mutual exclusion) ve
- senkronizasyon (synchronization)

- Burada temel nokta üreticinin tampon bellek miktarını aşmaksızın data üretmesi gerekirken, tüketicinin ise 0 data miktarının altında kalmaksızın datayı tüketmesi gerektiği gerçeğidir.

Semafor ile Üretici-Tüketici Problemi Çözümü

- Semafor kullanarak üretici tüketici probleminin, Bounded Buffer durumunda çözümü.
- Üretici ve tüketici birer process olarak oluşturulmaktadır.

- Burada çözüm için üç semafor kullanılmaktadır.

- semaphore mutex = 1; // kritik bölge karşılıklı dışarlama
- semaphore empty = BufferSize; // Buffer'daki boş yer sayısı
- semaphore full = 0; // Buffer'daki dolu yer sayısı

Semafor ile Üretici-Tüketici Problemi Çözümü

- Üretici ve tüketici **full** ve **empty** isimli semaforları senkronizasyon amacıyla kullanacaklardır.
- Eğer buffer dolu ise üretici tüketicin buffer'dan veri almasını bekler. **full** isimli semafor, bufferdaki dolu yuvaların sayısını saymak için kullanılır.
- Eğer buffer boş ise tüketici üreticinin buffer'a veri girmesini bekler. **empty** isimli semafor, bufferdaki dolu yuvaların sayısını saymak için kullanılır.
- Buffer'a aynı anda sadece bir process erişebilir. Bu maksatla **mutex** semaforu karşılıklı dışlama (mutual exclusion) için kullanılır.

Semafor ile Üretici-Tüketici Problemi Çözümü

```
# define N 100
# define TRUE 1                          / * Buffer'daki eleman sayısı */
typedef int semaphore;
semaphore mutex = 1;                     / * Semaforlar int bir tür olarak tanımlanıyor */
semaphore empty = N;                     / * kritik kısmı karşılıklı dışarlamak */
semaphore full = 0;                      / * Buffer'daki boş yer sayısı */
producer()                               / * Buffer'daki dolu yer sayısı */
{
    int item;
    while (TRUE) {                       / * yerel değişken */
        produce_item(&item);
        down(empty);                     / * Buffer'a konulacak veri oluşturuluyor */
        down(mutex);                     / * Buffer dolu ise bekle yoksa bos yer sayısını 1 azalt
        enter_item(item);                / * Kritik kısma girebilmek için vize al */
        up(mutex);                       / * Veriyi Buffer'a gir (Kritik Kısım) */
        up(full);                        / * Kritik Kısımdan çıktığını belirt */
                                         / * Bekleyen tüketici varsa uyandır yoksa Buffer'daki */
    }                                    / * dolu yer sayısını 1 arttır */

}
consumer(){
    int item;
    while (TRUE) {                       / * yerel değişken */
        down(full);
        down(mutex);                     / * Buffer boş ise bekle yoksa dolu yer sayısını 1 azalt */
        remove_item(&item);              / * Kritik kısma girebilmek için vize al */
        up(mutex);                       / * Veriyi Buffer'dan al (Kritik Kısım) */
        up(empty);                       / * Kritik Kısımdan çıktığını belirt */
                                         / * Bekleyen üretici varsa uyandır yoksa Buffer'daki*/
        consume_item(item);              / * dolu boş sayısını 1 arttır */
    }                                    / * Buffer'dan alınan veriyi kullan */
}
```

Semafor ile Üretici-Tüketici Problemi Çözümü

```
BufferSize = 3;
 semaphore mutex = 1;            // Controls access to critical section
 semaphore empty = BufferSize;    // counts number of empty buffer slots
 semaphore full = 0;             // counts number of full buffer slots
 Producer()
 {
  int widget;
  while (TRUE) {                 // loop forever
   make_new(widget);             // create a new widget to put in the buffer
   down(&empty);                 // decrement the empty semaphore
   down(&mutex);                 // enter critical section
   put_item(widget);             // put widget in buffer
   up(&mutex);                   // leave critical section
   up(&full);                    // increment the full semaphore
   }
 }
 Consumer()
 {
  int widget;
   while (TRUE) {                // loop forever
   down(&full);                  // decrement the full semaphore
   down(&mutex);                 // enter critical section
   remove_item(widget);          // take a widget from the buffer
   up(&mutex);                   // leave critical section
   up(&empty);                   // increment the empty semaphore
   consume_item(widget);         // consume the item
   }
 }
```

Okuyucular – Yazıcılar Problemi (Readers-Writers Problem)

- Birçok insan arasında paylaşılan bir dosyamızın olduğu bir durumu düşünün.

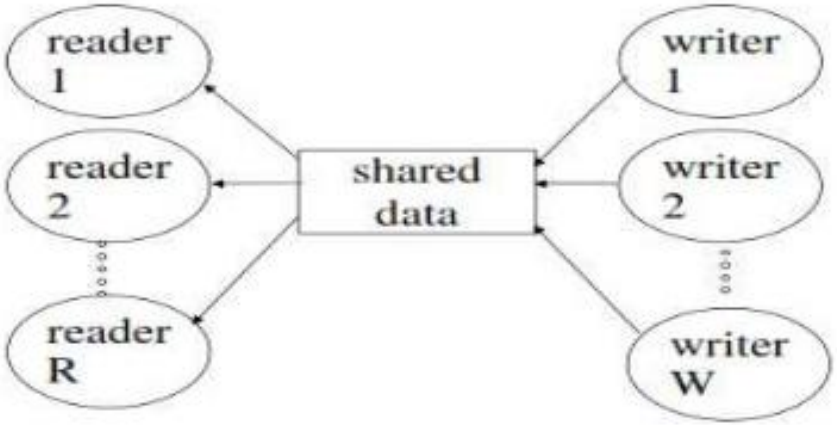
- İnsanlardan biri dosyayı düzenlemeye çalışırken, başka bir kişi aynı anda okuma veya yazma yapmamalıdır, aksi takdirde değişiklikler ona görünmeyecektir.

- Ancak, bir kişi dosyayı sadece okuyorsa, diğerleri bunu aynı anda okuyabilir.

Okuyucular – Yazıcılar Problemi (Readers-Writers Problem)

- Okuyucular ve Yazıcılar problemi, paylaşılan sınırlı bir kaynak için yarış durumunda olan (race condition) process'leri modellemek için kullanışlıdır.
- Bir kaynak, aynı anda çalışan birden fazla process arasında paylaştırılıyor.

- Okuyucular (Readers), sadece veriyi okuyorlar, veriyi güncellemiyorlar.
- Yazıcılar (Writers), hem okuyup hem de yazabiliyorlar.



Okuyucular – Yazıcılar Problemi (Readers-Writers Problem)

- Okuyucular ve yazıcılar probleminin pratik bir örneği, verileri okuyan ve yazabilen birçok process içeren büyük bir veri tabanından oluşan bir havayolu rezervasyon sistemidir.
- Hiçbir veri değişmediğinden veri tabanındaki bilgilerin okunması bir soruna yol açmayacaktır.
- Sorun, veri tabanına veri yazarken yatar.
- Veri tabanına herhangi bir kısıtlama getirilmezse, veriler her an değişebilir.
- Bir okuma process'i kullanıcıya veri talebi sonucunu görüntülediğinde, veri tabanındaki gerçek veriler değişmiş olabilir.
- Örneğin, bir process, bir uçuşta mevcut koltukların sayısını okur, bir değeri bulur ve müşteriye bildirir. Daha sonra müşterinin rezervasyon yapma şansı olmadan önce, başka bir process başka bir müşteri için rezervasyon yaparsa, mevcut koltuk sayısını değiştirir.

Okuyucular – Yazıcılar Problemi (Readers-Writers Problem)

- **Problem:** Verinin birden çok process tarafından okunduğu, ancak üzerine eş zamanlı veri yazımıyla sonuçların tutarsızlaşması durumu.
- **Çözüm:** Aynı anda birden fazla okuyucu process'e okumak için izin vermek, fakat aynı anda sadece bir yazıcı process'e paylaşılan veri kümesine yazma için izin vermek.

- Yazma işini yapacak process kaynağı kilitler ve yazılmasını engeller. İşini bitiren process çıkarken kilidi kaldırır ve kaynak diğer process'ler tarafından yazılabilir hale geçer.

Okuyucular – Yazıcılar Problemi (Readers-Writers Problem)

- Bir veri birden fazla process arasında paylaşılmaktadır.
- Bir kerede sadece bir yazıcı yazabilir.
- Bir process yazıyorsa, başka hiçbir process aynı anda onu okuyamaz.
- En az bir okuyucu okuyorsa, başka hiçbir process yazamaz.

Okuyucular – Yazıcılar Problemi (Readers-Writers Problem)

- **Varyasyon 1 :** Okuyucu process beklemez iken, yazıcı process, veri üzerinde işlem yapmak için izin istemek zorundadır. Bir yazıcı veriye erişim hakkını kazanmadığı sürece, hiçbir okuyucu beklemez.

- **Varyasyon 2:** Eğer bir yazıcı veriye erişmek için bekliyorsa, hiçbir yeni okuyucu okuma işlemine başlayamaz. Yazıcılar öncelikli.

- İki şekilde de okuyucu ve yazıcı process'ler beklemek (starvation) zorunda kalacaklardır.
- İlk varyasyonda yazıcı process'ler gecikebilir.
- İkinci varyasyonda okuyucu process'ler gecikebilir.

Varyasyon 1
Okuyucu Yazarın Üzerinde Öncelikli Olduğunda Çözüm

- Çözümü uygulamak için üç değişken kullanılır: mutex, wrt, readcnt
- **semaphore mutex, wrt;**
- // semaphore wrt: reader ve writer process'ler için ortak.
- // semaphore mutex : readcnt güncellendiğinde karşılıklı dışlamayı sağlamak için kullanılır, yani herhangi bir okuyucu kritik bölüme girdiğinde veya çıkarsa ve semafor wrt hem okuyucu hem de yazıcılar tarafından kullanılırsa.
- **int readcnt;**
- // readcnt, kritik bölgede okuma yapan process sayısını bildirir.

- Başlangıç değerleri
  - mutex=1; wrt=1; readcnt=0

Writer process:

```
do {
   // writer requests for critical section
   wait(wrt);

   // performs the write

   // leaves the critical section
   signal(wrt);

} while(true);
```

Yazıcı kritik bölgeye girme isteğinde bulunur.

<u>İzin verilirse</u>, yani wait(wrt) semaforu true değer verirse, yazıcı kritik bölgeye girer ve yazma işlemini gerçekleştirir.
Yazıcı kritik bölgeden çıkar.
signal(wrt) ile wrt semaforunun değeri değiştirilir.

<u>İzin verilmezse</u> beklemeye devam eder.

Reader process:

```
do {
   // Reader wants to enter the critical section
    wait(mutex);
    // The number of readers has now increased by 1
    readcnt++;
    // there is atleast one reader in the critical section
    // this ensure no writer can enter if there is even one reader
    // thus we give preference to readers here
     if (readcnt==1)
       wait(wrt);
    // other readers can enter while this current reader is inside
    // the critical section
   signal(mutex);
    // current reader performs reading here
    wait(mutex);   // a reader wants to leave
    readcnt--;
     // that is, no reader is left in the critical section,
     if (readcnt == 0)
       signal(wrt);       // writers can enter
     signal(mutex); // reader leaves
} while(true);
```

Okuyucu kritik bölgeye girmek ister.

<u>İzin verilirse:</u>

Kritik bölge içindeki okuyucu process sayısını artırır. Bu okuyucu ilk giren okuyucu ise, yazıcıların girişini kısıtlamak için wrt semaforunu kilitler.

Daha sonra, bu okuyucu kritik bölgede iken diğer okuyucuların da girmesine izin verilir.
Okuma yaptıktan sonra kritik bölümden çıkılır. Çıkarken, kritik bölgede daha fazla okuyucunun bulunup bulunmadığını kontrol eder, eğer başka okuyucu yoksa wrt semaforunun kilidi açılır ve yazıcılar kritik bölgeye girebilir.

<u>İzin verilmezse :</u> beklemeye devam eder.

Filozofların Akşam Yemeği (Dining philosopher)

- Akşam yemeği yiyen filozoflar örneği, literatüre Dijkstra tarafından kazandırılmıştır ve eş zamanlı process yönetimini (concurrent process management) sembolize eder.

- İkiden fazla filozof, bir yuvarlak masanın etrafına dizilerek yemek yerler.
- Literatürde örnek iki şekilde anlatılmaktadır ve orijinalinde filozoflar pirinç pilavı yemektedir.
- Buna göre pirinç yemek için iki adet yemek sopası (chopstick) gerekmektedir.
- Çinlilerin yemek yerken kullandıkları sopaları düşünebilirsiniz.
- Tek sopa ile yemek yenmesi imkansızdır ve her filozofun en az iki sopaya ihtiyacı vardır. Başlangıç durumunda, her filozofun iki yanında birer sopa durmaktadır.

Filozofların Akşam Yemeği
(Dining philosopher)

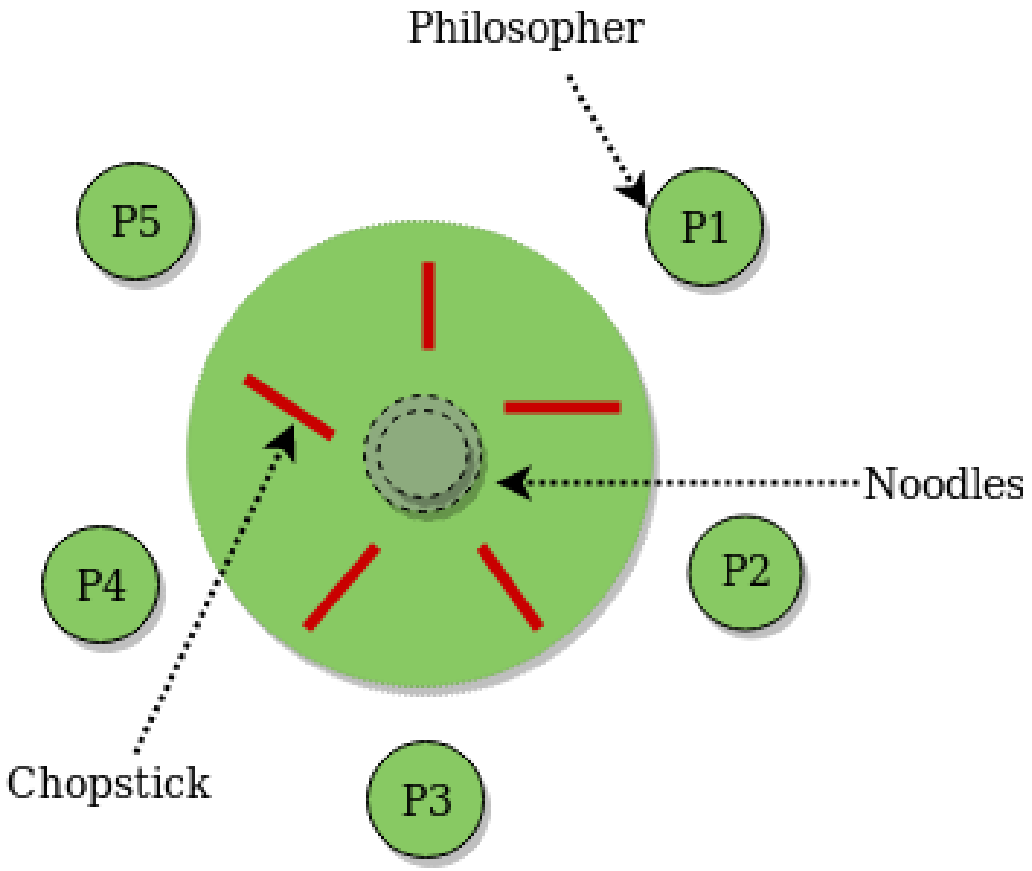
- Örneğin biraz daha batı dünyasına uyarlanmış halinde (buna göre batıda çatal kullanılmaktadır), her filozof makarna yemek ister ancak makarna yemek için en az iki çatala ihtiyaç vardır.

Filozofların Akşam Yemeği (Dining philosopher)

Masada 5 tane filozof oturduğunu ve sadece 5 tane yemek çubuğunu (veya çatal) bulunduğu düşünülüyor.

Filozoflar iki durumda olabiliyor; **düşünme** ve **yemek yeme** durumları. Düşünüyorlar, acıkınca da yemek yiyorlar. Döngü bu şekilde devam ediyor.

Bir filozof, ona bitişik durumda bulunan iki yemek çubuğunu (veya çatalı) alabilirse yemek yiyebilir.



Filozofların Akşam Yemeği (Dining philosopher)

- <u>Çözülmesi gereken problem :</u>
- Tüm filozoflar aynı anda yemek yemeye kalkarsa sofrada yeterince çatal yok.
- Buna göre filozofların hepsi örneğin sağındaki çatalı alırsa, hepsinde birer çatal olacak ve yemek yiyemeyeceklerdir.

Filozofların Akşam Yemeği (Dining philosopher)

- Şayet hepsi iki yanındakini birden almaya kalkarsa, bu durumda, eş zamanlı işlemlerde karşılaşılan yarış durumu (racing condition) ortaya çıkacaktır ve hangisi önce davranırsa o yemeğini yiyebilecektir.

- Yada hepsi birer çatal alacağı için yine hiçbirisi yemek yiyemeyecektir.

Filozofların Akşam Yemeği (Dining philosopher)

- Şayet hepsi birden çatallarını bir diğeri yesin diye bırakırsa, bu durumda yine hiçbirisi yemek yiyemeyecektir.

- Buna göre her filozofun yemek yeme ihtimali bulunmaktadır ancak hiçbir şekilde yiyeceği garanti edilmemektedir.

- Örneğin filozoflardan birisi her durumda aç kalabilir ve asla sıra kendisine gelmeyebilir.

Filozofların Akşam Yemeği (Dining philosopher)

- Problemde karşılaşılan diğer bir sorun ise ölümcül kilitlenmedir (deadlock).
- Yanlış bir tasarım sonucunda, tek çatal alan ve çatalı bırakmak için diğer filozofun bırakmasını bekleyen bir filozof sistemi kilitleyebilir.
- Bu da problemde bulunan ikinci risktir.

Filozofların Akşam Yemeği (Dining philosopher)

- Problemin tanımında, filozoflar birbiri ile konuşamaz kuralı getirilmiştir.
- Bu kural önemli bir kural olmakla birlikte, çözümlerin çoğunda bu kuralın ihlal edildiği görülebilir.
- Aslında filozoflar birbiriyle çatallar üzerinden iletişim kurmaktadır.
- Örneğin sağındaki veya solundaki filozofun o anda çatalı alıp almaması, yanındaki filozoflar hakkında bilgi vermekte ve bu da üstü kapalı bir iletişim olarak kabul edilmektedir.

Filozofların Akşam Yemeği
(Dining philosopher)

- Filozof 2 durumdadır: Düşünüyor (çatal gerekmez) ve Yiyor (çatal gerekir).
- Herhangi bir anda, bir filozof ya yemek yiyor ya da düşünüyordur.
- Bir filozof yemek istediğinde, iki tane çatal kullanır - biri soldan, diğeri sağdan.
- Bir filozof düşünmek istediğinde, her iki çatalı da orijinal yerlerinde tutar.
- Amaç filozofların yeme-düşünme döngüsünü sağlamaktır.

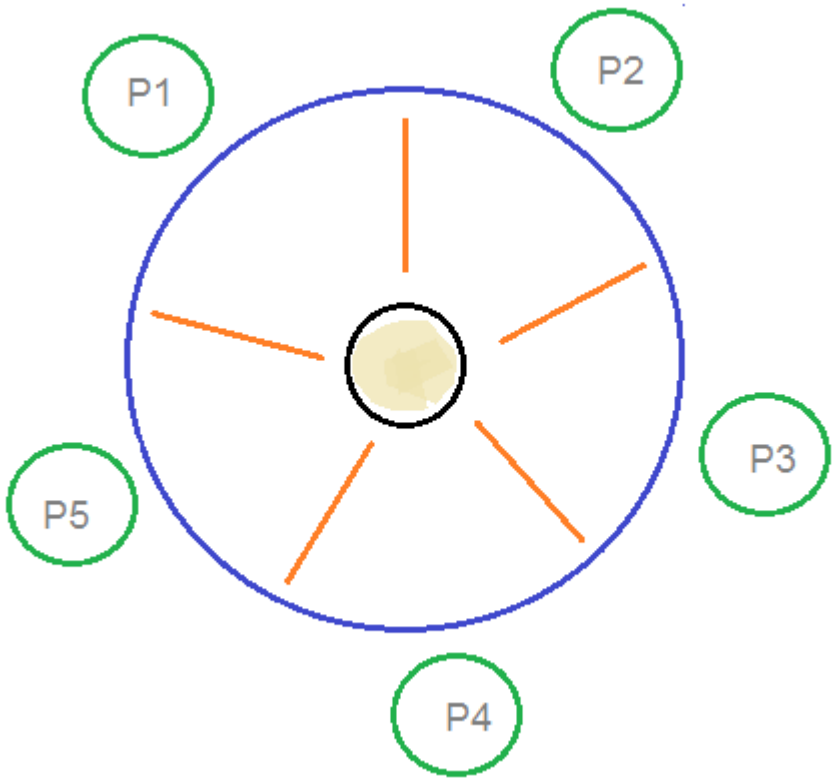
Filozofların Akşam Yemeği (Dining philosopher)

- Problem ifadesinden, bir filozofun belirsiz bir süre için düşünebileceği açıktır.
- Ama bir filozof yemeye başladığında, bir noktada da durması gerekir.
- Filozoflar sonsuz düşünme ve yeme döngüsünde kalmamalıdır.
- Ölümcül kilitlenme (deadlock) ve açlık durumu (starvation) oluşmamalı.
- Bu yüzden;
  - Filozofun, yalnızca sol ve sağ çatalları mevcutsa almasına izin verilmeli.
  - Sadece dört filozofun masaya oturmasına izin verilebilir. Böylelikle, dört filozofun dört çatal seçmesi durumunda, masanın üzerinde bir çatal kalır. Yani, bir filozof yemeye başlayabilir ve en sonunda iki çatal hazır olur. Bu şekilde, deadlock'lardan kaçınılabilir.

Filozofların Akşam Yemeği (Dining philosopher)

- Her filozof aşağıdaki sözde kod ile temsil edilir:
- Başlangıç değeri 1 olan bir dizi semafor kullanılır **stick[0..4]**

```
while(TRUE)
{
   wait(stick[i]);
   /* mod is used because if i=5, next
      chopstick is 1 (dining table is circular) */
   wait(stick[(i+1) mod 5]);

    /* eat */

    signal(stick[i]);
      signal(stick[(i+1) mod 5]);

     /* think */
}
```



Filozofların Akşam Yemeği (Dining philosopher)

- Problemin çözümü için farklı algoritmalar geliştirilmiştir.

- **1. Rastgele süre çözümü (Random Solution)**
- **2. Garson çözümü (Conductor Solution)**
- **3. Monitör Çözümü (Monitor Solution)**
- **4. Chandy Misra Çözümü (Chandy Misra Solution)**

Uyuyan Berber Problemi (Sleeping Barber)

- Bir berber dükkanında N sandalyeli bekleme odası ve berberin oturduğu berber odası vardır.
- Eğer müşteri yoksa berber uyur.
- Eğer bütün sandalyeler doluyken dükkana müşteri gelirse, müşteri geri gider.
- Berber meşgulse ama oturacak yer varsa gelen müşteri boş bir yere oturur.
- Berber uyuyorsa gelen müşteri berberi uyandırır.

Uyuyan Berber Problemi (Sleeping Barber)

- Çözüm:
- Problem tanımında bulunan ve senkronize edilmesi gereken durumları bulalım. Öncelikle problemimizde karşılıklı dışlama (mutual exclusion) problemi bulunmaktadır.

- Problem tanımındaki birden fazla process'in birbiri ile yarış durumuna (racing condition) girmesi halinde bazı kaynaklarda problem yaşanabilir.

Uyuyan Berber Problemi (Sleeping Barber)

- Örneğin aynı sandalyeye birden fazla müşterinin oturması veya berber koltuğuna birden fazla müşterinin oturması veya bir müşteri berber koltuğuna diğer müşteri kalkmadan oturmaya çalışırsa bu durumda yaşanacak problemler gibi çok sayıda problem bulunmaktadır.

- Bu problemlerin en hızlı ve doğru çözümü karşılıklı dışlama şeklinde processlerin senkronize edilmesidir.

- Diğer bir deyişle aynı anda tek bir işlemin çalışmasına izin verilirse problem çözülür.

Uyuyan Berber Problemi (Sleeping Barber) Semafor Şablonu

```
semaphore customer; init(customer,0); // hizmet bekleyen müşteri sayısı
semaphore barber; init(barber,0); // müşteri bekleyen berber sayısı
semaphore mutex; init(mutex,1); // karşılıklı dışlama için
int waiting = 0; //sandalyelerde oturan müşteriler


void barberThread(){
     while (1) {
          ……————————.
          cutHair();
          ————————————.
     }
}


void customerThread(){
     ————————————————..
     getHairCut(); // bütün sandalyeler doluysa çalıştırılmayabilir
     ————————————————
}
```

Uyuyan Berber Problemi (Sleeping Barber)

- B: Müşteri yokken berber uyuyor mu?
- BM: İlk müşteri berberi uyandırıp saçını kestiriyor mu?
- BMM: İkinci müşteri berberin ilk müşterinin tıraşını bitirmesini bekliyor mu?
- BMMMMMMM: 7. müşteri tıraşını olmadan ayrılır mı?
- BMMMMM: Berber ilk müşterinin tıraşını bitirdikten sonra bekleyen bir müşteriyi uyandırıyor mu?
- Son olarak bu çözüm verimli midir? İhtiyaç duyulandan fazla bildirim sinyali gönderme veya daha fazla değişken kullanma iyi bir uygulama değildir.